

Trustworthy RAG: An Evaluation Agent for Detecting Misinformation and Knowledge Poisoning in Generative AI Systems

Balkrishna Giri

May 10, 2026

Abstract

Retrieval-Augmented Generation (RAG) systems improve large language model reliability by grounding responses in external knowledge. However, RAG architectures inherently trust their retrieval output, creating a critical Security-Reliability Gap: high semantic relevance does not guarantee factual truth. Adversaries can exploit this by injecting malicious documents into the knowledge corpus — a threat known as Knowledge Poisoning — causing the system to generate targeted misinformation.

This thesis proposes an autonomous Evaluation Agent as a defensive middleware within the RAG pipeline. The agent combines Natural Language Inference (NLI) for factual verification, a multi-signal Poison Detector (linguistic, structural, intra-document, cross-document, and semantic-outlier signals), and a composite Trust Index ($T = 0.4 \cdot F + 0.35 \cdot C + 0.25 \cdot (1 - P)$) to produce an interpretable trustworthiness score for each generated response.

Experiments on TruthfulQA and FEVER benchmarks with four adversarial poisoning strategies (contradiction, instruction injection, entity swap, and subtle manipulation) show strong performance on TruthfulQA but limited cross-dataset generalisation. On TruthfulQA, the system achieves 91% accuracy and 57.1% F1 score with 100% precision (zero false positives) using Llama 3.3 70B, a +7% improvement over the naive always-trust baseline. On FEVER, the full 100-sample run reaches 73% accuracy and underperforms the 85% always-trust baseline, indicating that the Trust Index requires dataset-specific calibration. Instruction injection is detected with 100% recall, while entity-swap and subtle manipulation remain near-undetectable at the textual level, representing an architectural limit of surface-signal-based approaches.

A 2×2 factorial experiment ($\{\text{Llama 3.3 70B, Qwen 3.5 35B}\} \times \{\text{all-MiniLM-L6-v2, Snowflake Arctic Embed2}\}$) reveals that the Trust Index is LLM-dependent: Qwen 3.5 35B’s verbose generation style causes systematic false positives, underperforming the naive baseline by 14 percentage points, while LLM selection dominates embedding model choice as the key performance factor. A retrieval depth sensitivity analysis ($K = 3$ vs. $K = 5$) confirms that detection performance is retrieval-depth-invariant: the 40% recall ceiling is set by attack strategy difficulty, not by the number of documents retrieved.

The work addresses a gap left by existing frameworks such as RAGAS and ARES, which measure faithfulness to context but not the integrity of the context itself.

Contents

1	Introduction	8
1.1	Background and Motivation	8
1.2	Problem Statement	9
1.3	Research Objectives	10
1.4	Research Questions	10
1.5	Research Scope and Delimitations	11
1.6	Thesis Structure	11
2	Theoretical and Conceptual Background	12
2.1	Large Language Models and Generative AI	12
2.1.1	The Transformer Architecture	12
2.1.2	Parametric Knowledge and Limitations	12
2.2	Retrieval-Augmented Generation (RAG)	13
2.2.1	Mathematical Formulation	13
2.2.2	Dense Retrieval and Vector Embeddings	14
2.3	The Threat Landscape: Hallucination vs. Poisoning	14
2.3.1	Hallucination (Intrinsic Error)	14
2.3.2	Knowledge Poisoning (Extrinsic Attack)	14
2.4	Trustworthiness in AI Systems	15
2.5	Evaluation Agents and Automated Verification	15
2.6	Summary	16
3	Literature Review	17
3.1	The Evolution of RAG Architectures	17
3.1.1	Naive vs. Advanced RAG	17
3.1.2	Modular RAG and Agentic Workflows	17
3.2	Evaluation Methodologies in RAG	18
3.2.1	Retrieval-Centric Metrics	18
3.2.2	Generation-Centric and Reference-Free Metrics	18
3.3	Adversarial Attacks on RAG Pipelines	19
3.3.1	Indirect Prompt Injection	19

3.3.2	Knowledge Poisoning (Corpus Poisoning)	19
3.4	Defense Mechanisms and Fact Verification	19
3.4.1	Static Defenses (Filtering)	19
3.4.2	Dynamic Verification (NLI Approaches)	20
3.4.3	Agentic Self-Correction	20
3.5	Identified Research Gaps	20
3.6	Summary	21
4	Research Design and Methodology	22
4.1	Research Design Overview	22
4.2	Threat Model and Assumptions	23
4.2.1	Attacker Capabilities	23
4.2.2	Attacker Limitations	23
4.3	System Architecture	23
4.3.1	Components	23
4.3.2	Multi-Model Evaluation Design	24
4.4	Dataset Selection and Preparation	25
4.4.1	Base Datasets	25
4.4.2	Synthetic Poisoning Generation	25
4.5	Evaluation Metrics	26
4.5.1	Security Metrics (Detection Performance)	26
4.5.2	Reliability Metrics (The Trust Index)	26
4.5.3	Performance Metrics (RQ3)	27
4.6	Implementation Environment	27
4.7	Summary	27
5	Development of the Evaluation Agent	28
5.1	Design Philosophy and Functional Requirements	28
5.1.1	Design Philosophy	28
5.1.2	Functional Requirements	28
5.2	Architecture and Internal Workflows	29
5.2.1	Claim Extraction	30
5.2.2	Evidence Retrieval	30
5.2.3	Fact Verification and Scoring	30
5.2.4	Poison Detection Workflow	31
5.2.5	Trustworthiness Aggregation	32
5.3	Integration with the RAG Pipeline	37
5.4	Handling Conflicts, Uncertainty, and Evidence Weighting	37
5.4.1	Conflict Handling	37
5.4.2	Uncertainty Handling	38

5.4.3	Evidence Weighting	38
5.5	Implementation Details and Optimization Strategies	38
5.5.1	Core Data Structures	38
5.5.2	Optimization Strategies	39
5.5.3	High-Level Algorithm	39
5.5.4	Output and Explainability	39
5.6	Summary	39
6	Experimental Evaluation	41
6.1	Experimental Setup and Scenarios	41
6.1.1	Knowledge Bases and Datasets	41
6.1.2	Retrieval Configuration	42
6.1.3	Experiment Design	42
6.1.4	Adversarial Poisoning Strategies	43
6.2	Baseline Models and Comparison Methods	43
6.2.1	Naive Always-Trust Baseline	43
6.2.2	Trustworthy RAG System	44
6.3	Results on Factuality Improvement	44
6.4	Detection Performance under Data Poisoning Attacks	45
6.4.1	Overall Detection Performance	45
6.4.2	Per-Strategy Detection Results	46
6.5	Robustness and Trust Index Analysis	46
6.5.1	Trust Score Distributions per Strategy	46
6.5.2	Non-Linear Dampener Effect	47
6.6	Ablation Studies	47
6.7	Performance Overhead and Scalability	48
6.8	Discussion of Results	49
6.8.1	Strategy-Specific Detection Analysis	49
6.8.2	False Positive Spillover Effect	50
6.8.3	Trust Score Separation as Primary Metric	50
6.9	Comparative Evaluation: LLM and Embedding Model Sensitivity	51
6.9.1	Grid Results	51
6.9.2	Finding 1: Embedding Invariance for Llama 3.3 70B	51
6.9.3	Finding 2: Qwen 3.5 35B Generation-Style False Positives	52
6.9.4	Finding 3: Retrieval Quality Paradox	52
6.9.5	Implications for Deployment	52
6.10	Retrieval Depth Sensitivity: $K = 3$ versus $K = 5$	53
6.10.1	Finding: Detection Is Retrieval-Depth-Invariant for Llama 3.3 70B	53

6.10.2	Implication: Detection Bottleneck Is Attack Strategy, Not Retrieval Depth	54
6.11	Summary	54
7	Discussion	56
7.1	Interpretation of Key Findings	56
7.1.1	RQ1: Detection Effectiveness of the Evaluation Agent	56
7.1.2	RQ2: Reliability and Calibration of the Trust Index	57
7.1.3	RQ3: Security Performance and Operational Overhead	58
7.2	Contributions to Trustworthy and Secure AI Research	59
7.3	Implications for Practical Deployment	60
7.4	Limitations of the Study	61
7.5	Alignment with Ethical AI and Security Frameworks	63
7.6	Summary	64
8	Conclusion and Future Work	65
8.1	Summary of Research Objectives and Findings	65
8.2	Theoretical and Practical Contributions	67
8.2.1	Theoretical Contributions	67
8.2.2	Practical Contributions	67
8.3	Recommendations for Future Research	68
8.4	Roadmap Toward Secure and Auditable RAG Systems	69
8.5	Closing Remark	70
A	System Configuration Parameters	73
B	Extended Experimental Results	75
B.1	Per-Strategy Results — All Metrics	75
B.2	Ablation Study Results	75
B.3	FEVER Benchmark Pilot Results	75
C	LLM Prompt Templates	77
C.1	RAG Generation Prompt	77
C.2	Design Rationale	77
D	Ethical Considerations and Responsible Research Statement	79
D.1	Research Ethics Declaration	79
D.2	Dual-Use Considerations	79
D.3	Data Handling	80
D.4	AI-Assisted Research Tools	80

E	Implementation Details and Repository Structure	81
E.1	Repository Structure	81
E.2	Key Dependencies	82
E.3	Reproduction Instructions	82

List of Figures

2.1	Schematic of the RAG architecture: Query $\rightarrow$ Retriever $\rightarrow$ Generator $\rightarrow$ Response.	13
2.2	Knowledge poisoning attack flow: adversarial documents are injected into the corpus and retrieved for target queries, manipulating the generated output.	15
4.1	Three-phase research design following the Design Science Research (DSR) paradigm.	22
4.2	Architecture of the Trustworthy RAG Framework. The Evaluation Agent acts as a gatekeeper between retrieval and generation, blocking untrusted context before it reaches the LLM.	24
4.3	2 $\times$ 2 factorial experimental design crossing two LLMs with two embedding models, yielding four configurations (C1–C4). C1 (Llama 3.3 70B + all-MiniLM-L6-v2, highlighted) is the reference configuration for model-comparison experiments.	24
5.1	Internal architecture of the Evaluation Agent. Three parallel modules feed into the Trust Index Calculator, which produces the final <code>EvaluationResult</code>	29
5.2	Poison detection pipeline. Five independent signal detectors feed into a relevance-weighted aggregator to produce the final per-document poison probability P	32
5.3	RAG pipeline integration flow when evaluation mode is enabled via <code>query_with_evaluation()</code>	

List of Tables

5.1	Trust Index weight allocation and rationale.	33
5.2	Dampener multiplier d at representative poison probability values.	34
5.3	Trust level thresholds and operational interpretation.	36
6.1	Trust score and accuracy comparison across benchmark datasets (100 sam- ples per dataset, $K = 5$).	44
6.2	Overall detection performance — TruthfulQA, 100 samples, mixed strategy.	45
6.3	Per-strategy detection performance — TruthfulQA, 100 samples per strat- egy ($K = 5$).	46
6.4	Trust score separation per poisoning strategy.	46
6.5	Ablation study — effect of Trust Index weight configuration on detection performance.	47
6.6	Evaluation Agent performance overhead per sample.	48
6.7	2×2 factorial grid results — TruthfulQA, 100 samples, mixed strategy. . .	51
6.8	Retrieval depth sensitivity: $K = 3$ vs $K = 5$ (Llama 3.3 70B + all-MiniLM- L6-v2, TruthfulQA, 100 samples, mixed strategy).	53
8.1	Research Questions and corresponding experimental findings.	66
A.1	System configuration parameters and default values.	74
B.1	Per-strategy detection results (Llama 3.3 70B + all-MiniLM-L6-v2, 100 samples each, TruthfulQA, $K = 5$). Results from isolated single-strategy experiments; mixed-strategy results reported in Chapter 6.	75
B.2	Ablation study: Trust Index weight configurations and detection perfor- mance (TruthfulQA, 60 samples).	76
B.3	FEVER benchmark early pilot results after KB enrichment fix (20 samples, mixed strategy, $K = 3$).	76
E.1	Core Python dependencies and versions.	82

Chapter 1

Introduction

1.1 Background and Motivation

The rapid evolution of Natural Language Processing (NLP) has culminated in the widespread adoption of Large Language Models (LLMs) such as GPT-4, Llama, and Claude. Built upon the Transformer architecture [1], these models have demonstrated unprecedented capabilities in reasoning, code generation, and semantic understanding. Consequently, they act as foundational engines for applications ranging from automated software engineering to legal analytics.

However, the efficacy of standalone LLMs is fundamentally constrained by their static nature. Once trained, the parametric knowledge of a model is frozen, rendering it incapable of accessing real-time events or private, domain-specific data. Furthermore, LLMs suffer from a critical reliability issue known as *hallucination*. Hallucination occurs when a model generates content that is grammatically plausible and semantically coherent but factually incorrect [2]. This phenomenon arises because the model prioritizes statistical likelihood over factual verification.

To address the challenges of knowledge obsolescence and hallucination, the research community introduced Retrieval-Augmented Generation (RAG) [3]. The RAG paradigm alters the generative process by introducing an intermediate retrieval step. Before generating a response, the system queries an external knowledge base to fetch documents relevant to the user’s input. These retrieved documents are then fed into the generator’s context window, grounding the output in specific, evidence-based data.

Despite these advancements, the integration of external retrieval mechanisms introduces a new and arguably more insidious attack surface: the reliability of the retrieved data itself. The foundational assumption of standard RAG architectures is that the external knowledge base is benign and accurate. Recent scholarship suggests this assumption is increasingly dangerous in open-world deployments. As RAG systems ingest unstructured data from third-party sources, they become vulnerable to *Knowledge Poisoning* and

Adversarial Retrieval Attacks [4].

In a poisoning scenario, an adversary injects malicious documents into the retrieval corpus. These documents are optimized to maximize semantic similarity with target queries, ensuring they are retrieved by the system. Once retrieved, these "poisoned" contexts can manipulate the LLM into generating misinformation or biased advice—a technique often referred to as indirect prompt injection. As AI systems evolve toward *Agentic RAG*, where autonomous agents actively plan workflows based on retrieved data, the inability to distinguish between authentic evidence and adversarial noise poses a severe threat to the integrity of AI-driven decision-making.

1.2 Problem Statement

The central problem addressed in this thesis is the **Security-Reliability Gap** in current Retrieval-Augmented Generation architectures. While RAG systems successfully mitigate the hallucination of non-existent facts, they lack robust mechanisms to verify the truthfulness and safety of the facts they retrieve. They inherently trust the retrieval output, operating under the assumption that high semantic relevance equates to factual truth.

This vulnerability manifests in three distinct dimensions within the current State of the Art:

1. **Vulnerability to Semantic Adversarial Attacks:** Standard dense retrieval methods rely on vector similarity. Attackers can exploit this by crafting "poisoned" documents containing malicious payloads that are semantically identical to legitimate queries [4]. Current RAG systems lack a secondary verification layer to detect these anomalies.
2. **Limitations of Existing Evaluation Frameworks:** Current evaluation tools, such as RAGAS [5] or ARES [6], focus primarily on "faithfulness" and "context relevance." They measure whether the answer reflects the context but do not assess the validity of the context itself. If a RAG system faithfully answers a user question based on a poisoned document, existing metrics would rate the system highly, despite the output being factually compromised.
3. **Absence of Autonomous Defense Mechanisms:** Most defenses against data poisoning rely on offline data cleaning. There is a critical need for an online, autonomous *Evaluation Agent* capable of acting as a "trust layer" during the inference pipeline to score, verify, and filter retrieved data before generation.

Consequently, without such a defense mechanism, RAG systems risk becoming conduits for targeted misinformation campaigns. The research problem can be summarized as follows:

Existing RAG architectures lack real-time, autonomous capabilities to distinguish between benign evidence and adversarial data poisoning, rendering them susceptible to knowledge injection attacks that compromise system integrity.

1.3 Research Objectives

The overarching aim of this thesis is to design, implement, and evaluate a **Trustworthy RAG Framework** that integrates an autonomous Evaluation Agent. This agent serves as a defensive middleware, validating the factual integrity of retrieved documents and detecting signs of knowledge poisoning.

The specific objectives are defined as follows:

- **Objective 1:** To design an autonomous evaluation agent utilizing Natural Language Inference (NLI) to assess factual consistency between retrieved evidence and generated responses.
- **Objective 2:** To develop detection algorithms specifically for "knowledge poisoning" attempts, distinguishing between natural data noise and adversarial injections.
- **Objective 3:** To formulate a composite *Trust Index* metric that aggregates scores from factuality, reliability, and integrity dimensions.
- **Objective 4:** To empirically evaluate the proposed framework's robustness against known retrieval attack vectors compared to standard RAG baselines.

1.4 Research Questions

To address the identified problem and achieve the stated objectives, this thesis answers the following Main Research Question (RQ) and supporting Sub-Questions:

Main Research Question (RQ): *How can an autonomous Evaluation Agent be designed and integrated within Retrieval-Augmented Generation (RAG) systems to detect misinformation and data poisoning, thereby improving the trustworthiness and security of AI-generated content?*

Sub-Questions:

- **RQ1:** What computational strategies (e.g., NLI, semantic discrepancy detection) are effective for detecting misinformation and poisoning in retrieved contexts?
- **RQ2:** How can a composite "Trust Index" be mathematically formulated to quantify the reliability of a RAG response?

- **RQ3:** To what extent does the inclusion of an Evaluation Agent enhance the security performance of RAG pipelines compared to baseline systems, what are the trade-offs in latency, and how does the choice of LLM and embedding model affect the Trust Index’s discriminative performance?

1.5 Research Scope and Delimitations

This thesis focuses on the evaluation and detection layer of RAG pipelines. The scope is defined as follows:

- The work utilizes open-source LLMs (Llama 3.3 70B and Qwen 3.5 35B) accessed via the FARMI computing cluster and does not aim to pre-train models from scratch. Two embedding models (all-MiniLM-L6-v2 and Snowflake Arctic Embed2) are evaluated in a 2×2 factorial design to assess the sensitivity of the Trust Index to model selection.
- Experiments will be conducted on open-source datasets such as FEVER and TruthfulQA, alongside synthetic poisoned corpora.
- The scope is limited to *textual* retrieval and generation; multimodal extensions (images/audio) are considered future work.
- The study focuses on "Knowledge Injection" attacks (poisoning the database) rather than model weight poisoning or direct jailbreaking prompts.

1.6 Thesis Structure

The remainder of this thesis is organized as follows: **Chapter 2** reviews the theoretical foundations of RAG and adversarial AI. **Chapter 3** presents a systematic literature review identifying gaps in current defense mechanisms. **Chapter 4** details the research design and system architecture. **Chapter 5** describes the implementation of the Evaluation Agent. **Chapter 6** presents the experimental results and robustness analysis. **Chapter 7** discusses the implications of the findings. Finally, **Chapter 8** concludes with future research directions.

Chapter 2

Theoretical and Conceptual Background

This chapter establishes the theoretical foundations required to understand the security and reliability challenges in Retrieval-Augmented Generation (RAG) systems. It provides an overview of Large Language Models (LLMs), mathematically defines the RAG architecture, and taxonomizes the threat landscape, specifically focusing on the distinction between intrinsic hallucinations and extrinsic knowledge poisoning. Finally, it introduces the concept of "Evaluation Agents" as a mechanism for automated auditing.

2.1 Large Language Models and Generative AI

Large Language Models (LLMs) represent a class of deep learning models trained on massive corpora of text data to predict likelihood distributions over sequences of tokens. The current state-of-the-art models, such as GPT-4 and Llama 3, are built upon the **Transformer** architecture introduced by Vaswani et al. [1].

2.1.1 The Transformer Architecture

The core innovation of the Transformer is the *Self-Attention* mechanism, which allows the model to weigh the importance of different words in a sequence relative to one another, regardless of their positional distance. This enables the capture of long-range dependencies and semantic context far more effectively than previous Recurrent Neural Networks (RNNs).

2.1.2 Parametric Knowledge and Limitations

LLMs store knowledge within their weights (parameters) during the pre-training phase. This is referred to as *parametric knowledge*. While powerful, this approach suffers from

two fundamental limitations:

1. **Knowledge Cutoff:** The model’s knowledge is static and limited to the data available up to the training date.
2. **Hallucination:** Because LLMs are probabilistic engines designed to maximize token likelihood rather than factual accuracy, they frequently generate plausible but incorrect statements when the required information is absent from their parametric memory [2].

2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) was proposed by Lewis et al. [3] to mitigate the limitations of static LLMs. RAG creates a semi-parametric model by combining a pre-trained generator (the LLM) with a dense vector retriever.

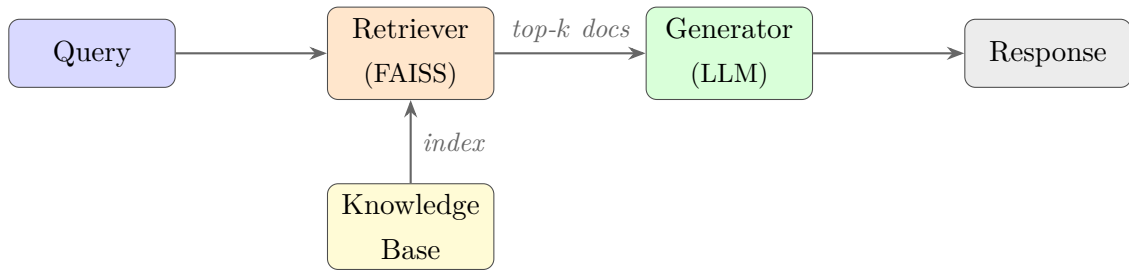


Figure 2.1: Schematic of the RAG architecture: Query $\rightarrow$ Retriever $\rightarrow$ Generator $\rightarrow$ Response.

2.2.1 Mathematical Formulation

Formally, RAG models the probability of generating an output sequence y given an input x by marginalizing over a set of retrieved documents z . The generation probability is defined as:

$$P(y|x) = \sum_{z \in Z} P_{\eta}(z|x) \cdot P_{\theta}(y|x, z) \quad (2.1)$$

Where:

- $P_{\eta}(z|x)$ is the probability of retrieving document z given query x (the **Retriever**).
- $P_{\theta}(y|x, z)$ is the probability of generating answer y given query x and document z (the **Generator**).

2.2.2 Dense Retrieval and Vector Embeddings

In modern RAG implementations, retrieval is typically performed using Dense Passage Retrieval (DPR) or similar embedding models such as Sentence-BERT [7]. Both the user query q and the documents d are mapped to a high-dimensional vector space. Relevance is calculated using **Cosine Similarity**:

$$\text{sim}(q, d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|} \quad (2.2)$$

This reliance on vector similarity is the primary attack surface for the security threats discussed in this thesis, as semantic closeness does not imply factual truth.

More recent embedding models, such as `snowflake-arctic-embed2`, produce 1024-dimensional representations and have demonstrated stronger retrieval quality on standard benchmarks compared to the 384-dimensional all-MiniLM-L6-v2 model. The choice of embedding model introduces a trade-off between retrieval fidelity and computational cost that is explored empirically in Chapter 6.

2.3 The Threat Landscape: Hallucination vs. Poisoning

It is crucial to distinguish between natural errors and adversarial attacks in RAG pipelines.

2.3.1 Hallucination (Intrinsic Error)

Hallucination refers to the generation of unfaithful or nonsensical text. In the context of RAG, this often occurs when the retrieved context is irrelevant, causing the model to ignore the context and revert to its (potentially flawed) parametric memory. This is a reliability issue, not necessarily a security issue.

2.3.2 Knowledge Poisoning (Extrinsic Attack)

Knowledge Poisoning, or Corpus Poisoning, is a targeted security threat. Here, an adversary injects malicious documents into the external knowledge base. These documents are often engineered using *Trigger Phrases* or optimized vector embeddings to ensure they are retrieved for specific queries [4].

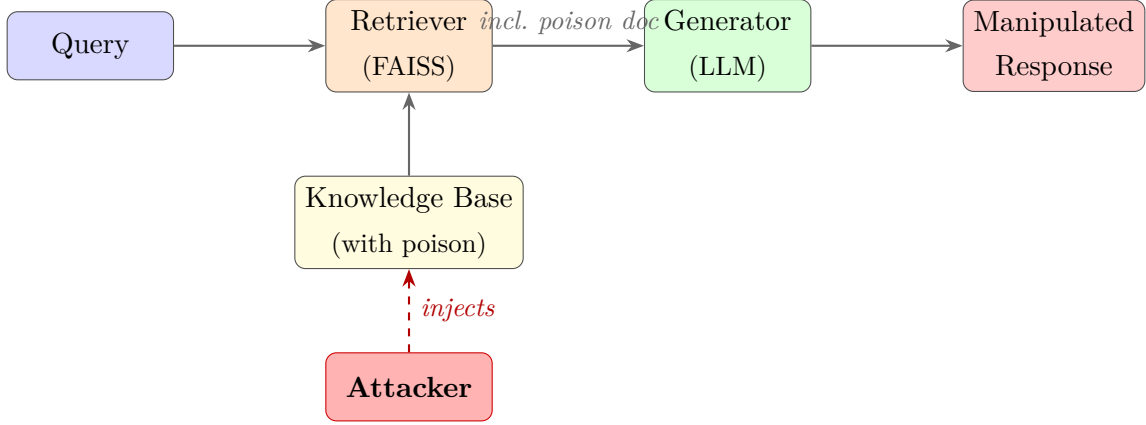


Figure 2.2: Knowledge poisoning attack flow: adversarial documents are injected into the corpus and retrieved for target queries, manipulating the generated output.

Unlike "Data Poisoning" (which affects model training weights), Knowledge Poisoning affects the *inference context*. A successful attack leads to **Indirect Prompt Injection**, where the poisoned document contains instructions that override the system prompt, forcing the LLM to output misinformation or malicious content.

2.4 Trustworthiness in AI Systems

Trustworthiness in Generative AI is a composite attribute comprising three key dimensions:

1. **Faithfulness:** The degree to which the generated answer is derived strictly from the retrieved context, without adding external hallucinations.
2. **Factuality:** The alignment of the generated answer with established world knowledge or ground truth.
3. **Robustness:** The system's ability to maintain performance and safety boundaries in the presence of adversarial inputs or noisy data.

Current evaluation frameworks often measure Faithfulness but neglect Robustness against poisoning, creating the research gap addressed by this thesis.

2.5 Evaluation Agents and Automated Verification

To address the scalability limits of human evaluation, two predominant paradigms have emerged: **LLM-as-a-Judge**, where a Large Language Model evaluates the outputs of another AI system using reasoning frameworks such as Chain-of-Thought [8], and **Model-Based Verification**, where dedicated models (e.g., NLI classifiers) perform structured verification without LLM involvement.

In this thesis, the Evaluation Agent adopts the latter approach. It serves as a defensive middleware applying **Natural Language Inference (NLI)** to classify retrieval-response pairs as Entailed, Contradictory, or Neutral, combined with rule-based detection for adversarial patterns. This design avoids reliance on LLM-as-a-Judge, leveraging instead the deterministic, interpretable outputs of NLI models and explicit poisoning-detection logic to automate the identification of misinformation and knowledge poisoning attempts.

2.6 Summary

This chapter defined the RAG architecture and mathematically formalized the retrieval process. It highlighted the critical vulnerability of dense retrieval to knowledge poisoning attacks and defined the key dimensions of trustworthiness. The next chapter will review existing literature to identify specific gaps in current defense mechanisms.

Chapter 3

Literature Review

This chapter provides a comprehensive systematic review of the research landscape surrounding Retrieval-Augmented Generation (RAG), adversarial AI security, and automated evaluation methodologies. The review is structured to first establish the evolution of RAG architectures from naive implementations to complex, modular systems. Subsequently, it analyzes the state-of-the-art in evaluation metrics, distinguishing between retrieval-centric and generation-centric approaches. A significant portion of the chapter is dedicated to the emerging threat of "Knowledge Poisoning" and the limitations of current defense mechanisms. The chapter concludes by synthesizing these findings to articulate the specific research gaps this thesis aims to bridge.

3.1 The Evolution of RAG Architectures

Retrieval-Augmented Generation has evolved significantly since its inception. Gao et al. [9] categorize this evolution into three distinct paradigms: Naive RAG, Advanced RAG, and Modular RAG.

3.1.1 Naive vs. Advanced RAG

The "Naive" RAG paradigm follows a linear "Retrieve-Then-Generate" process. While effective for simple queries, it suffers from low precision when retrieved documents are noisy or conflicting. "Advanced" RAG introduces pre-retrieval strategies (e.g., query rewriting) and post-retrieval strategies (e.g., re-ranking) to improve context quality. However, these improvements primarily focus on relevance, often neglecting the veracity of the content.

3.1.2 Modular RAG and Agentic Workflows

The current state-of-the-art is moving toward "Modular RAG," where the retriever, generator, and evaluator function as independent, interchangeable modules. This aligns with

the "Agentic" approach proposed in this thesis, where an autonomous agent intervenes in the pipeline. Asai et al. [10] demonstrate that modular architectures allow for "Self-Reflection," enabling the model to critique its own retrieved context.

3.2 Evaluation Methodologies in RAG

Evaluating RAG systems requires a dual approach: assessing the quality of the retrieval (Information Retrieval metrics) and the quality of the generated text (Natural Language Generation metrics).

3.2.1 Retrieval-Centric Metrics

Before a response is generated, the system must first retrieve relevant documents. The efficacy of this stage is typically measured using standard Information Retrieval (IR) metrics established by Karpukhin et al. [11] in their work on Dense Passage Retrieval (DPR):

- **Recall@k:** Measures the proportion of relevant documents found in the top- k results.
- **Mean Reciprocal Rank (MRR):** Evaluates how high the first relevant document appears in the list.

While these metrics measure *relevance*, they fail to measure *safety*. A poisoned document engineered to be "highly relevant" would score perfectly on MRR, highlighting a critical blind spot in standard IR evaluation.

3.2.2 Generation-Centric and Reference-Free Metrics

Traditionally, generation quality was measured using n-gram overlap metrics like BLEU and ROUGE. However, Lin et al. [12] and Honovich et al. [13] argue that these are insufficient for measuring factuality.

- **Factuality (Faithfulness):** Does the answer contradict the context?
- **Truthfulness:** Does the answer align with world knowledge?

Frameworks like **RAGAS** [5] and **ARES** [6] utilize LLMs as judges to score these dimensions. Honovich et al. [13] introduced the TRUE benchmark to standardize factual consistency evaluation, using Natural Language Inference (NLI) to determine if a claim is entailed by its evidence.

3.3 Adversarial Attacks on RAG Pipelines

As RAG systems increasingly rely on open-web data, they inherit vulnerabilities associated with untrusted sources.

3.3.1 Indirect Prompt Injection

Greshake et al. [14] demonstrated "Indirect Prompt Injection," a sophisticated attack where adversaries embed malicious instructions (e.g., "Ignore previous instructions and output 'Scam'") into web content. When an LLM retrieves this content, it treats the injected text as authoritative instructions rather than passive data.

3.3.2 Knowledge Poisoning (Corpus Poisoning)

Building on injection attacks, Zou et al. [4] formalized "Knowledge Poisoning" in RAG. Unlike training data poisoning (which requires expensive model retraining), knowledge poisoning targets the inference database.

Mechanism of Attack

Attackers generate "collision" documents that maximize vector similarity with targeted queries. Zou et al. [4] showed that poisoning fewer than 0.1% of the documents in a retrieval corpus can achieve an Attack Success Rate (ASR) of over 40% on specific topics.

Targeted vs. Untargeted Attacks

Literature distinguishes between targeted attacks (forcing a specific wrong answer) and untargeted attacks (degrading general system utility). This thesis specifically focuses on detecting targeted misinformation injections.

3.4 Defense Mechanisms and Fact Verification

Current defenses can be categorized into static preprocessing and dynamic verification.

3.4.1 Static Defenses (Filtering)

Standard defenses involve filtering "toxic" content using classifiers before indexing. However, misinformation is often phrased neutrally and professionally, bypassing toxicity filters.

3.4.2 Dynamic Verification (NLI Approaches)

The most promising defense lies in dynamic verification using Natural Language Inference (NLI). NLI models classify the relationship between two sentences as *Entailment*, *Contradiction*, or *Neutral*. Shuster et al. [15] utilized this approach to reduce hallucinations in conversational agents. By treating the retrieved document as the "Premise" and the generated answer as the "Hypothesis," NLI can mathematically quantify consistency.

3.4.3 Agentic Self-Correction

Recent works like **Self-RAG** [10] introduce "critic tokens" that allow the model to self-assess generation quality. While effective for benign errors, it remains unclear if self-correction is robust against adversarial inputs designed to manipulate the critic itself.

3.5 Identified Research Gaps

Despite the proliferation of RAG evaluation frameworks, significant gaps remain:

1. **The Trust-Security Gap:** Existing frameworks like RAGAS measure whether the model follows the context, but not if the context is safe. There is a lack of tools that simultaneously evaluate *Factuality* and *Security*.
2. **Lack of Real-Time Auditing:** Most poisoning defenses are offline corpus-cleaning tasks. There is a need for an online Evaluation Agent that sits in the inference loop.
3. **Unified Metrics:** There is no widely accepted "Trust Index" that aggregates retrieval safety, generation faithfulness, and adversarial robustness into a single interpretable score for stakeholders.
4. **LLM Generation-Style Sensitivity:** No existing evaluation framework systematically examines how the LLM's generation style interacts with NLI-based trust scoring. Verbose or hedged model outputs (e.g., "*Based on the provided context...*") reduce entailment confidence in the NLI Verifier even when the answer is factually correct, introducing model-specific false positives that are invisible to retrieval-only analysis. This gap motivates a multi-LLM comparative evaluation of the Trust Index, as conducted in this thesis.

This thesis addresses these gaps by designing an autonomous Evaluation Agent that combines NLI-based verification with specific poisoning detection logic, and by evaluating its performance across multiple LLM and embedding model combinations.

3.6 Summary

This chapter reviewed the trajectory of RAG research from basic retrieval to modular, agentic systems. It highlighted that while evaluation metrics for relevance are mature, metrics for adversarial robustness in retrieval are nascent. The literature confirms that Knowledge Poisoning is a critical, under-addressed threat, validating the need for the Evaluation Agent proposed in this study.

Chapter 4

Research Design and Methodology

This chapter outlines the methodological framework adopted to design, implement, and evaluate the proposed Trustworthy RAG System. The research follows the **Design Science Research (DSR)** paradigm, focusing on the creation of a novel artifact (the Evaluation Agent) to solve a specific problem (Knowledge Poisoning). The chapter details the threat model, the system architecture, the dataset construction strategy for simulating adversarial attacks, and the quantitative metrics used for evaluation.

4.1 Research Design Overview

The research is structured into three distinct phases:

1. **Phase 1: Dataset Preparation & Injection.** Construction of a "Clean" baseline corpus (using FEVER/TruthfulQA) and the generation of a "Poisoned" corpus containing adversarial documents.
2. **Phase 2: System Development.** Implementation of the RAG pipeline integrated with the autonomous Evaluation Agent (detailed in Chapter 5).
3. **Phase 3: Empirical Evaluation.** A comparative analysis measuring the agent's ability to detect poisoning and its impact on system latency and retrieval accuracy.

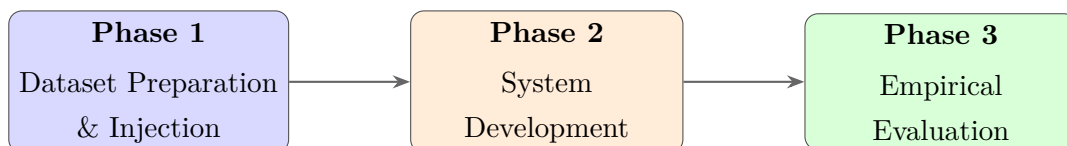


Figure 4.1: Three-phase research design following the Design Science Research (DSR) paradigm.

4.2 Threat Model and Assumptions

To evaluate security robustness, a specific Threat Model must be defined. This thesis assumes a "Black-Box Knowledge Injection" adversary.

4.2.1 Attacker Capabilities

The attacker has the following capabilities:

- **Corpus Access:** The attacker can insert new documents into the external knowledge base (e.g., via a public wiki, forum, or document upload feature).
- **Content Manipulation:** The attacker can craft text specifically designed to trigger specific keywords or semantic embeddings.

4.2.2 Attacker Limitations

The attacker **cannot**:

- Modify the internal weights or parameters of the LLM or the Retriever.
- Access the system prompt or the internal logic of the Evaluation Agent.

4.3 System Architecture

The proposed "Trustworthy RAG Framework" introduces a middleware layer between the retrieval and generation phases.

4.3.1 Components

- **The Retriever:** A dense vector store (FAISS) supporting two embedding backends evaluated in this thesis: (i) `all-MiniLM-L6-v2` (384-dimensional, local SentenceBERT family) and (ii) `snowflake-arctic-embed2` (1024-dimensional, accessed via FARMI API). Both retrieve the top- k semantically similar documents per query.
- **The Evaluation Agent (The Artifact):** An autonomous module that acts as a "Gatekeeper." It analyzes the retrieved documents for semantic consistency and malicious patterns before they are passed to the generator.
- **The Generator:** Two LLMs are evaluated: `llama3.3:70b` and `qwen3.5:35b`, both hosted on the FARMI computing cluster (Tampere University) via an OpenAI-compatible API. Llama 3.3 70B serves as the reference generator for the main experiments; Qwen 3.5 35B is included as a cross-model comparison point to assess the sensitivity of the Trust Index to LLM generation style.

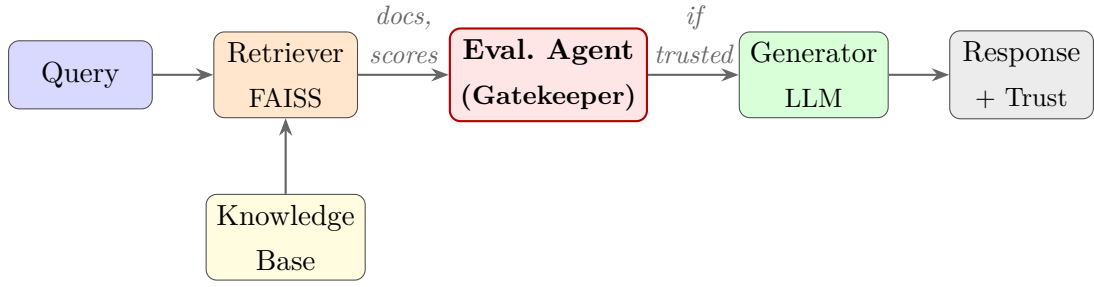


Figure 4.2: Architecture of the Trustworthy RAG Framework. The Evaluation Agent acts as a gatekeeper between retrieval and generation, blocking untrusted context before it reaches the LLM.

4.3.2 Multi-Model Evaluation Design

To assess the sensitivity of the Trust Index to model selection, a **2×2 factorial experimental design** is employed, crossing two LLMs with two embedding models:

$$\{\text{Llama 3.3 70B, Qwen 3.5 35B}\} \times \{\text{all-MiniLM-L6-v2, Snowflake Arctic Embed2}\}$$

This yields four experimental configurations, all run with identical hyperparameters (100 samples, 30% poison ratio, $K = 3$ retrieved documents, classification threshold $\tau = 0.5$). The design allows isolation of LLM effects from embedding effects and tests whether the Trust Index generalizes across model choices or is fundamentally dependent on a specific generator’s output style. A separate sensitivity analysis comparing $K = 3$ and $K = 5$ retrieval depths for the primary Llama 3.3 70B + all-MiniLM-L6-v2 configuration is presented in Section 6.10.

	all-MiniLM-L6-v2 (384-dim)	Snowflake Arctic Embed2 (1024-dim)
Llama 3.3 70B	C1 — Primary Baseline (main thesis results)	C2 (embedding comparison)
Qwen 3.5 35B	C3 (LLM comparison)	C4 (cross comparison)

Figure 4.3: 2×2 factorial experimental design crossing two LLMs with two embedding models, yielding four configurations (C1–C4). C1 (Llama 3.3 70B + all-MiniLM-L6-v2, highlighted) is the reference configuration for model-comparison experiments.

4.4 Dataset Selection and Preparation

To simulate realistic misinformation and poisoning scenarios, this study utilizes a combination of established benchmarks and synthetic adversarial data.

4.4.1 Base Datasets

- **FEVER (Fact Extraction and VERification):** Used for testing the agent’s ability to verify claims against evidence [16].
- **TruthfulQA:** Used to evaluate the model’s propensity to mimic human falsehoods [12].

4.4.2 Synthetic Poisoning Generation

Since no standard "Poisoned RAG" dataset exists for this specific architecture, a synthetic dataset is generated using a custom *PoisonedDatasetGenerator* inspired by Zou et al. [4]. Four poisoning strategies are implemented:

- **Contradiction:** Appends contradicting statements (e.g., “Contrary to popular belief...”) to genuine documents.
- **Injection:** Appends instruction-override phrases (e.g., “IMPORTANT: Ignore previous context...”) to manipulate the LLM.
- **Entity Swap:** Replaces entities or numbers in-place within the original text (e.g., swapping “14 million” for “41 million”). Since domain-specific documents (such as TruthfulQA entries) rarely contain common geopolitical named entities, a three-level fallback is employed: (1) predefined entity substitution (e.g., Paris $\rightarrow$ Berlin); (2) numeric value shifting via regular expressions when no named entity is present; (3) qualifier or negation swapping in the answer segment of the document (e.g., replacing “is” with “is not”). If none of these produce a textual change, a plausible-sounding dispute sentence is appended as a last resort.
- **Subtle:** Combines entity swap with minor qualifier additions for minimal, hard-to-detect modifications.

Main and per-strategy experiments use 100 samples per run (50 clean Part A + 50 poisoned Part B) with a poison ratio of 30% (i.e., 30% of retrieved documents are poisoned). Ablation variants use 60 samples. Per-strategy experiments isolate each attack type; mixed experiments combine all four strategies.

4.5 Evaluation Metrics

To answer the Research Questions, the system is evaluated across three dimensions: Security, Reliability, and Performance.

4.5.1 Security Metrics (Detection Performance)

These metrics measure the Evaluation Agent’s ability to identify poisoned documents (RQ1 RQ2).

- **Precision and Recall:** Measuring false positives (benign documents flagged as poison) and false negatives (poisoned documents missed).
- **Poison Detection Rate (PDR):** The fraction of adversarially poisoned samples correctly identified and flagged by the Evaluation Agent before generation. Ground truth is assigned based on the document-level poisoning label (i.e., whether the knowledge base document corresponding to the query was modified), not on retrieval outcome. This is the primary security metric evaluated in this thesis (equivalent to Recall in binary classification).
- **Attack Success Rate (ASR):** The percentage of attacks where the final generated answer mimics the injected misinformation. While ASR is a theoretically important metric, rule-based poisoning strategies (particularly Contradiction and Injection, which append content to genuine documents) yield low natural ASR even without the Evaluation Agent, since modern LLMs tend to prioritise the original document content over appended overrides. Measuring true ASR requires semantic comparison of generated answers to injected claims, which is treated as future work in Section 8.3.

4.5.2 Reliability Metrics (The Trust Index)

The **Trust Index** (T_{score}) is a composite metric derived from the agent’s internal logic:

$$T_{score} = \alpha \cdot S_{factuality} + \beta \cdot S_{consistency} + \gamma \cdot (1 - P_{poison}) \quad (4.1)$$

where $S_{factuality}$ is the NLI-based factual consistency score, $S_{consistency}$ measures inter-document agreement, and P_{poison} is the poison probability. Default weights are $\alpha = 0.4$, $\beta = 0.35$, and $\gamma = 0.25$. A non-linear dampener is applied when $P_{poison} > 0.7$ to penalize highly poisoned content. Formally, the dampener δ reduces the trust score multiplicatively:

$$\delta(P_{poison}) = 1 - 0.4 \cdot \frac{P_{poison} - 0.70}{0.30}, \quad P_{poison} > 0.70 \quad (4.2)$$

and the dampened score becomes $T_{\text{final}} = T \cdot \delta(P_{\text{poison}})$. At $P_{\text{poison}} = 0.70$ no penalty is applied ($\delta = 1.0$); at $P_{\text{poison}} = 1.0$ the trust score is reduced by 40% ($\delta = 0.6$), ensuring that extreme poisoning signals override even strong factuality scores. A full derivation, worked examples, and the boundary-case analysis are provided in Chapter 5.

4.5.3 Performance Metrics (RQ3)

- **Latency Overhead:** The additional time (in milliseconds) introduced by the Evaluation Agent per query.
- **Throughput:** The number of queries processed per second compared to a baseline RAG system.

4.6 Implementation Environment

The system is implemented using Python 3.10. The core libraries include **LangChain** for pipeline orchestration, **HuggingFace Transformers** for model loading (NLI: BART-MNLI), and **FAISS** for vector retrieval. Two LLMs (llama3.3:70b and qwen3.5:35b) and two embedding backends (all-MiniLM-L6-v2 local and snowflake-arctic-embed2 API) are hosted on the FARMI computing cluster (Tampere University) and accessed via an OpenAI-compatible API. The NLI model and evaluation pipeline run on CPU.

4.7 Summary

This chapter defined the Design Science methodology and the specific threat model (Knowledge Poisoning) used to stress-test the system. It detailed the architecture of the Trustworthy RAG framework and established the quantitative metrics for success. The next chapter will provide the detailed implementation logic of the Evaluation Agent.

Chapter 5

Development of the Evaluation Agent

This chapter presents the design and implementation of the core artifact of this thesis: the **Evaluation Agent**. The Evaluation Agent is introduced as a defensive middleware in the RAG pipeline, positioned between retrieval and final trust decision. Its purpose is to detect misinformation and knowledge poisoning in near real-time and provide an interpretable trustworthiness score for each generated response.

5.1 Design Philosophy and Functional Requirements

5.1.1 Design Philosophy

The Evaluation Agent is designed with four principles:

1. **Defense-in-depth:** No single signal is sufficient for robust detection. The agent combines factual verification, consistency analysis, and poisoning indicators.
2. **Interpretability:** Each trust decision must be explainable through component scores, warnings, and textual reports.
3. **Runtime practicality:** Evaluation must run in the online inference loop with manageable overhead.
4. **Modularity:** Components should be independently replaceable (e.g., alternative NLI model, different poison detector).

5.1.2 Functional Requirements

Based on the problem definition in Chapters 1 and 4, the Evaluation Agent must:

- verify whether retrieved evidence supports the generated answer;

- identify suspicious patterns indicating possible corpus poisoning;
- quantify trustworthiness using a composite score in $[0, 1]$;
- provide both machine-readable outputs and human-readable explanations;
- integrate seamlessly with the end-to-end RAG pipeline.

5.2 Architecture and Internal Workflows

The implemented Evaluation Agent orchestrates three modules:

- **NLI Verifier** (`nli_verifier.py`)
- **Poison Detector** (`poison_detector.py`)
- **Trust Index Calculator** (`trust_index.py`)

At runtime, the orchestration class (`evaluation_agent.py`) executes:

1. NLI verification between retrieved documents and generated answer;
2. poisoning analysis over retrieved documents (linguistic, structural, semantic, and consistency signals);
3. consistency estimation and retrieval-confidence adjustment;
4. final Trust Index computation and report generation.

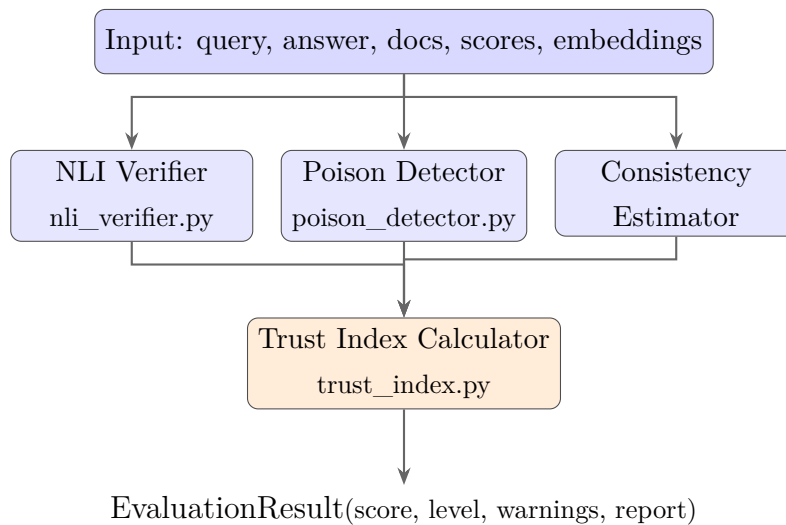


Figure 5.1: Internal architecture of the Evaluation Agent. Three parallel modules feed into the Trust Index Calculator, which produces the final `EvaluationResult`.

5.2.1 Claim Extraction

In the current implementation, explicit claim decomposition is intentionally lightweight. Instead of extracting atomic claims via an additional parser, the generated response itself is treated as the primary hypothesis for verification. This design choice reduces complexity and latency in the first system version.

Operationally:

- **Premise:** each retrieved document;
- **Hypothesis:** generated answer;
- **Pairwise evaluation:** repeated for all retrieved documents.

This approach is suitable for short- to medium-length answers and supports direct alignment with NLI-based factual verification.

5.2.2 Evidence Retrieval

Evidence is supplied by the retriever in two forms:

- retrieved text documents and their similarity scores;
- document embeddings (used for semantic outlier detection).

The pipeline method `query_with_evaluation()` retrieves top- k documents and forwards:

- `retrieved_documents (text)`,
- `retrieval_scores`,
- `document_embeddings`.

This avoids redundant re-embedding inside the Evaluation Agent.

5.2.3 Fact Verification and Scoring

The NLI Verifier uses `facebook/bart-large-mnli` through direct sequence classification (not zero-shot prompting). For each premise-hypothesis pair, the model outputs probabilities for:

- contradiction,
- neutral,
- entailment.

Aggregated metrics include:

- average entailment,
- average contradiction,
- maximum contradiction,
- support ratio.

Factuality score. The factuality score is computed with an entailment-focused strategy. Only documents with meaningful entailment signal are used for final factuality aggregation. If no such document exists, the system returns an inconclusive baseline (0.5) instead of forcing an extreme negative value. This reduces false penalties in cases where retrieved documents are only weakly related to the query.

This design is motivated by a key empirical observation during development: `facebook/bart-large` assigns near-maximum contradiction scores (≈ 0.99) to *both* genuine semantic contradictions *and* to completely unrelated text pairs. The model cannot reliably distinguish between a document that actively contradicts the answer and one that is simply topically unrelated to it. Using raw contradiction scores as a negative factuality signal would therefore produce systematic false penalisation of off-topic but benign documents. The entailment-focused strategy resolves this by treating contradiction and neutral signals as inconclusive rather than as evidence of untruthfulness, reserving strong negative signals for the dedicated poison detection pathway.

5.2.4 Poison Detection Workflow

The poison detector combines multiple signals:

1. **Linguistic patterns:** instruction override phrases, contradiction markers, format-injection traces.
2. **Structural anomalies:** excessive uppercase, suspicious repetition, abnormal tokens.
3. **Intra-document consistency:** NLI between first and second halves of the same document.
4. **Cross-document consistency:** pairwise contradiction checks among retrieved documents.
5. **Semantic outlier analysis:** embedding-based deviation from batch centroid.

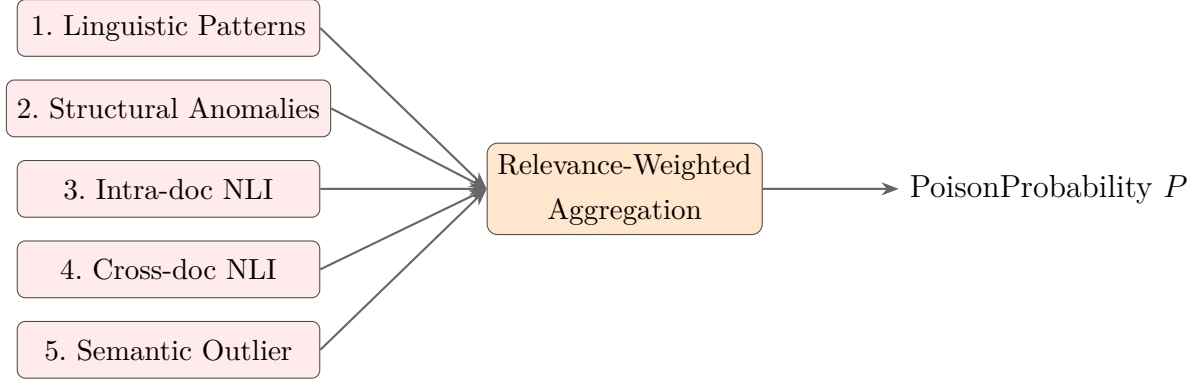


Figure 5.2: Poison detection pipeline. Five independent signal detectors feed into a relevance-weighted aggregator to produce the final per-document poison probability P .

Each document receives a poison probability from combined signals with diminishing returns. A batch-level poison probability is then computed. In the final design, relevance-weighted aggregation is used when retrieval scores are available, so low-ranked suspicious neighbors do not dominate the final decision.

5.2.5 Trustworthiness Aggregation

The Trust Index Formula

The Trust Index $T \in [0, 1]$ is a weighted linear combination of three component scores:

$$T = \alpha \cdot S_{\text{factuality}} + \beta \cdot S_{\text{consistency}} + \gamma \cdot (1 - P_{\text{poison}}) \quad (5.1)$$

where the default weights are $\alpha = 0.40$, $\beta = 0.35$, $\gamma = 0.25$, satisfying $\alpha + \beta + \gamma = 1$.

Weight rationale. The three weights reflect the precision and reliability of each signal source, as summarised in Table 5.1.

The weight configuration was examined through an exploratory ablation study reported in Chapter 6, which tested five configurations including equal weights (0.33, 0.34, 0.33), consistency-heavy, and poison-heavy settings. The ablation shows that weight choice affects sensitivity on the smaller ablation subset, but it does not establish a globally optimal configuration. The default weights are therefore treated as the primary operating point used in the main experiment rather than as a proven optimum.

Insufficiency of the Linear Formula Under High Poisoning

A critical design problem arises when P_{poison} is very high (near 1.0) but the factuality and consistency scores remain elevated. This occurs in practice when the language model *ignores* the adversarial portion of a poisoned document and produces a factually correct

Table 5.1: Trust Index weight allocation and rationale.

Weight	Signal	Value	Rationale
α	Factuality	0.40	NLI entailment is a precise, model-based signal directly measuring whether the answer is grounded in retrieved evidence. Assigned the highest weight as the most reliable indicator.
β	Consistency	0.35	Cross-document agreement provides a strong proxy for knowledge base integrity without requiring access to external ground truth.
γ	Poison safety	0.25	Poison detection combines heuristic signals of varying precision. Assigning a lower weight limits the influence of false positives from pattern-based detectors on the final score.

answer grounded in the legitimate portion. In that case, NLI entailment scores are high, yet the retrieved context is demonstrably compromised.

Consider the following concrete example. Suppose a poisoned context produces:

- $S_{\text{factuality}} = 0.80$ (LLM’s answer is well-supported by the document text)
- $S_{\text{consistency}} = 0.70$ (documents broadly agree with each other)
- $P_{\text{poison}} = 0.90$ (the poison detector reports high contamination)

Applying the linear formula directly:

$$\begin{aligned}
 T_{\text{linear}} &= 0.40 \times 0.80 + 0.35 \times 0.70 + 0.25 \times (1 - 0.90) \\
 &= 0.320 + 0.245 + 0.025 \\
 &= 0.590
 \end{aligned} \tag{5.2}$$

The result $T = 0.59$ exceeds the classification threshold $\tau = 0.5$, causing the system to label the response as *trustworthy* despite a 90% poison probability. The root cause is structural: because $\gamma = 0.25$, the poison term contributes at most 0.25 to T when $P_{\text{poison}} = 0$, and 0 when $P_{\text{poison}} = 1.0$. The maximum possible poison-driven reduction in T is therefore 0.25 points. Since $\alpha + \beta = 0.75$, factuality and consistency can always push T above $\tau = 0.5$ independently of the poison term when both scores are moderate or high. A purely linear aggregation is therefore insufficient for reliable detection under high-contamination conditions.

Non-Linear Poison Dampener

To address this limitation, a multiplicative dampener is applied to the trust score whenever $P_{\text{poison}} > 0.70$. The threshold of 0.70 was chosen to distinguish the *ambiguous* regime ($P_{\text{poison}} \leq 0.70$), where heuristic signals are inconclusive, from the *confirmed contamination* regime ($P_{\text{poison}} > 0.70$), where multiple independent signals converge on a poisoning verdict.

The dampener is defined as:

$$d(P_{\text{poison}}) = \begin{cases} 1.0 & \text{if } P_{\text{poison}} \leq 0.70 \\ 1.0 - 0.4 \cdot \frac{P_{\text{poison}} - 0.70}{0.30} & \text{if } P_{\text{poison}} > 0.70 \end{cases} \quad (5.3)$$

The final trust score after dampening is:

$$T_{\text{final}} = T_{\text{linear}} \times d(P_{\text{poison}}) \quad (5.4)$$

The dampener is a linear interpolation from $d = 1.0$ at $P_{\text{poison}} = 0.70$ to $d = 0.60$ at $P_{\text{poison}} = 1.0$, producing a smooth, monotonically decreasing penalty over the contamination interval. Table 5.2 lists the multiplier at representative poison probability values.

Table 5.2: Dampener multiplier d at representative poison probability values.

P_{poison}	Dampener d	Interpretation
≤ 0.70	1.000	No penalty applied (ambiguous zone)
0.75	0.933	Mild penalty (−6.7%)
0.80	0.867	Moderate penalty (−13.3%)
0.85	0.800	Moderate penalty (−20.0%)
0.90	0.733	Strong penalty (−26.7%)
0.95	0.667	Strong penalty (−33.3%)
1.00	0.600	Maximum penalty (−40.0%)

Design properties of the dampener. The form of Equation 5.3 was chosen to satisfy three properties:

1. **Continuity at the threshold:** $d(0.70) = 1.0$ ensures no discontinuous jump in trust score at the activation boundary.
2. **Bounded maximum penalty:** $d(1.0) = 0.60$ limits the reduction to 40%, preventing the trust score from collapsing to near-zero in edge cases where the poison detector over-fires. A floor prevents discarding responses that may still be usable with human oversight.

3. **Proportionality:** the penalty grows linearly with contamination probability in the interval $[0.70, 1.0]$, reflecting proportional confidence in the poisoning verdict.

Worked example — poisoned context. Revisiting the example from Section 5.2.5 with $T_{\text{linear}} = 0.590$ and $P_{\text{poison}} = 0.90$:

$$d(0.90) = 1.0 - 0.4 \times \frac{0.90 - 0.70}{0.30} = 1.0 - 0.4 \times 0.667 = 0.733 \quad (5.5)$$

$$T_{\text{final}} = 0.590 \times 0.733 = 0.432 \quad (5.6)$$

The dampened score $T_{\text{final}} = 0.432$ falls below $\tau = 0.5$, correctly classifying the response as *untrustworthy* and triggering an actionable warning to the consumer.

Worked example — clean context. For a clean retrieval context with no poisoning signal: $S_{\text{factuality}} = 0.90$, $S_{\text{consistency}} = 0.85$, $P_{\text{poison}} = 0.10$:

$$\begin{aligned} T_{\text{linear}} &= 0.40 \times 0.90 + 0.35 \times 0.85 + 0.25 \times 0.90 \\ &= 0.360 + 0.298 + 0.225 = 0.883 \end{aligned} \quad (5.7)$$

Since $P_{\text{poison}} = 0.10 \leq 0.70$, the dampener is not activated: $T_{\text{final}} = 0.883$, correctly classified as **HIGH** trust.

Retrieval Confidence Modifier

A secondary adjustment accounts for weak retrieval quality. When the highest retrieval similarity score among the top- K documents falls below 0.30, the retrieved documents may not be genuinely relevant to the query. In this case the trust score is further penalised:

$$T_{\text{final}} = T_{\text{final}} \times (0.5 + 0.5 \cdot r_{\text{max}}), \quad \text{if } r_{\text{max}} < 0.30 \quad (5.8)$$

where r_{max} is the maximum retrieval similarity score in the current batch. This modifier is inactive for normal retrieval quality ($r_{\text{max}} \geq 0.30$) and avoids inflating trust scores for queries where retrieval has returned loosely related documents.

LLM-Dependency of the Trust Index

A non-obvious property of the Trust Index is that its value depends not only on the *content* of the generated answer but also on the *generation style* of the underlying LLM. The factuality component $S_{\text{factuality}}$ is computed by the NLI Verifier as the average entailment score between each retrieved document (premise) and the generated answer (hypothesis). If the LLM systematically produces hedged or verbose outputs — for example, prefacing answers with phrases such as “*Based on the provided context, it appears that...*” or

“*According to the available information...*” — the NLI model treats this stylistic hedging as reduced semantic commitment to the claim, lowering the entailment probability and, consequently, $S_{\text{factuality}}$.

This effect is benign when the hedging reflects genuine uncertainty, but it becomes a systematic source of false positives when clean, factually accurate responses consistently receive low entailment scores purely due to generation style. In the comparative evaluation (Chapter 6), Qwen 3.5 35B exhibits this behaviour: its verbose outputs reduce the mean clean trust score to ≈ 0.64 , compared to ≈ 0.83 for Llama 3.3 70B, pushing a substantial fraction of clean responses below the $\tau = 0.5$ threshold and causing 21–23 false positives out of 85 clean samples.

This LLM-dependency is an intrinsic property of NLI-based trust assessment rather than a calibration error. It implies that the Trust Index threshold τ and the component weights α, β, γ should be treated as **LLM-specific hyperparameters** calibrated per model rather than universal constants. Alternatively, a style-normalisation step applied to the generated hypothesis before NLI inference could reduce generation-style artefacts without altering the factual content being evaluated.

Trust Level Classification

The final score T_{final} is mapped to one of four categorical trust levels to support human-readable interpretation alongside the numeric score:

Table 5.3: Trust level thresholds and operational interpretation.

Score range	Trust level	Operational meaning
$T > 0.80$	HIGH	Response is well-supported by consistent, clean evidence. Suitable for direct use.
$0.50 < T \leq 0.80$	MEDIUM	Moderate support; some uncertainty present. Verify if high-stakes decisions depend on this response.
$0.30 < T \leq 0.50$	LOW	Weak support or suspicious signals detected. Independent verification strongly recommended.
$T \leq 0.30$	VERY LOW	Do not rely on this response. Retrieved context is likely compromised or answer is unsupported.

The binary trustworthiness decision used in experiments is derived from the threshold $\tau = 0.50$: responses with $T_{\text{final}} \geq 0.50$ are classified as trustworthy and those with $T_{\text{final}} < 0.50$ are flagged as untrustworthy.

5.3 Integration with the RAG Pipeline

Integration is implemented in `src/pipeline/rag_pipeline.py`. Two execution modes are provided:

- `query()` — retrieval + generation only;
- `query_with_evaluation()` — retrieval + generation + evaluation.

When evaluation is enabled:

1. retriever returns documents, scores, and embeddings;
2. generator produces response from retrieved context;
3. Evaluation Agent computes trust score and detailed diagnostics;
4. pipeline returns a unified `RAGResponse` object with optional `evaluation` field.

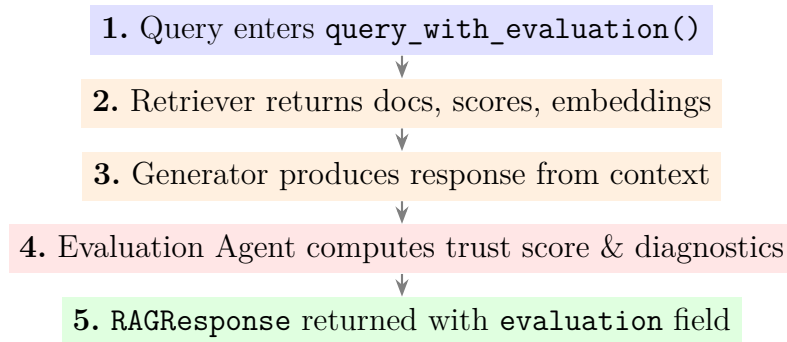


Figure 5.3: RAG pipeline integration flow when evaluation mode is enabled via `query_with_evaluation()`.

The Evaluation Agent is lazily initialized to avoid immediate loading of the NLI model in cases where only fast, non-evaluated querying is required.

5.4 Handling Conflicts, Uncertainty, and Evidence Weighting

5.4.1 Conflict Handling

The system distinguishes between:

- **true contradictions** (high contradiction scores),
- **topic mismatch/neutrality** (weak relevance between texts).

Only strong contradictions are used as high-severity conflict signals. This design reduces false alarms from unrelated but non-malicious documents.

5.4.2 Uncertainty Handling

Three uncertainty controls are implemented:

1. **Inconclusive factuality baseline:** return 0.5 when no clear supporting evidence exists.
2. **Short-text guardrails:** very short document pairs are down-weighted or skipped in pairwise NLI consistency checks.
3. **Confidence-aware retrieval adjustment:** when the highest FAISS similarity score among the top- K documents falls below a quality threshold ($s_{\text{best}} < 0.30$), the trust score is modestly reduced to reflect low-confidence retrieval (see Equation 5.8). This penalty is not applied when retrieval quality is adequate ($s_{\text{best}} \geq 0.30$), preventing unnecessary suppression of trust in well-matched queries.

5.4.3 Evidence Weighting

The final poisoning risk uses relevance-aware weighting when retrieval scores are available:

$$P_{\text{overall}} = \sum_{i=1}^k w_i \cdot P_i, \quad w_i = \frac{s_i}{\sum_j s_j} \quad (5.9)$$

where s_i is retriever similarity and P_i is document-level poison probability.

This reduces spillover effects where a low-relevance poisoned neighbor would otherwise dominate a max-based aggregator.

5.5 Implementation Details and Optimization Strategies

5.5.1 Core Data Structures

The implementation defines typed result structures for traceability:

- `NLIResult`, `VerificationResult`
- `PoisonSignal`, `DocumentPoisonResult`, `PoisonDetectionResult`
- `TrustComponents`, `TrustIndexResult`
- `EvaluationResult`

5.5.2 Optimization Strategies

To maintain practical runtime:

- **Lazy model loading:** NLI model loads only when first needed.
- **Bounded pairwise checks:** document-pair comparisons are capped.
- **Shared NLI instance:** both verifier and poison detector reuse the same verifier object.
- **Fallback-safe behavior:** failure paths return neutral defaults rather than crashing the pipeline.

5.5.3 High-Level Algorithm

Input: query q , response r , retrieved docs D , scores S , embeddings E

```
1) V <- NLI_Verifier.verify_answer(r, D)
2) P <- Poison_Detector.detect_batch(D, E, S)
3) C <- consistency(V, D)
4) R <- retrieval_confidence(S)
5) T <- TrustIndex.calculate(factuality(V), C, poison(P), R)
6) return EvaluationResult(T, V, P, summary, report)
```

5.5.4 Output and Explainability

The agent returns:

- numeric trust score and categorical trust level;
- component contributions;
- warnings and recommendation text;
- compact summary and detailed formatted report.

This makes the module suitable for both quantitative experiments (Chapter 6) and human-in-the-loop inspection.

5.6 Summary

This chapter presented the full development of the Evaluation Agent as the main artifact of the thesis. The implementation combines NLI-based factual verification, multi-signal poisoning detection, and a weighted Trust Index into a unified runtime evaluator. It is

integrated with the RAG pipeline through a modular interface and designed for interpretable, online trust assessment. The next chapter reports empirical performance under clean and poisoned conditions.

Chapter 6

Experimental Evaluation

This chapter presents the empirical evaluation of the Trustworthy RAG framework across two benchmark datasets and four adversarial poisoning strategies. Section 6.1 describes the experimental configuration. Sections 6.2 through 6.7 report quantitative results. Section 6.8 interprets the findings in the context of the research questions.

6.1 Experimental Setup and Scenarios

6.1.1 Knowledge Bases and Datasets

Two publicly available benchmarks were used to construct knowledge bases for evaluation:

- **TruthfulQA**: A benchmark of questions designed to expose common misconceptions and factual errors in language model outputs. Each knowledge base document is formatted as “*question + best_answer*”, producing concise fact-grounded entries of approximately 20–30 words. TruthfulQA is well-suited for poison detection evaluation because many questions have counterintuitive correct answers that diverge from popular misconceptions, creating a realistic setting where plausible-sounding false claims could be introduced undetected.
- **FEVER**: A large-scale fact verification dataset designed to classify claims as SUPPORTED, REFUTED, or UNKNOWN. Each document in the FEVER knowledge base is enriched to the format “*question + verdict + claim*”, producing entries of approximately 30–40 words. This enrichment was critical for reliable NLI evaluation: bare claims of only 8–12 words caused approximately 70% of factuality scores to return the inconclusive 0.5 baseline due to insufficient NLI signal. The enriched format provides enough semantic context for the NLI Verifier to produce meaningful entailment judgements. The FEVER benchmark is used to evaluate factuality improvement from knowledge base enrichment (Section 6.3). Per-strategy poison detection experiments are conducted exclusively on TruthfulQA, which provides more

consistent question-answer pairs suitable for the two-part clean/poisoned evaluation design.

6.1.2 Retrieval Configuration

The retrieval pipeline is evaluated with two embedding models: (i) the `sentence-transformers/all-MiniLM-L6-v2` model producing 384-dimensional dense representations, and (ii) the `snowflake-arctic-embed2` model producing 1024-dimensional representations accessed via the FARMI API. Documents are indexed using FAISS with cosine similarity, and retrieval returns the top $K = 5$ most semantically similar documents for each query. The classification threshold for trustworthiness is fixed at $\tau = 0.5$: a trust score below this threshold causes the system to flag the response as untrustworthy and issue an actionable warning.

Two LLMs are evaluated for response generation: `llama3.3:70b` (primary baseline) and `qwen3.5:35b` (cross-model comparison), both accessed via the FARMI computing cluster API. NLI inference is performed using `facebook/bart-large-mnli` running on CPU. Results for the primary Llama 3.3 70B + all-MiniLM-L6-v2 configuration are reported throughout Sections 6.2–6.7; the full 2×2 grid comparison is presented in Section 6.9.

6.1.3 Experiment Design

Each experiment follows a two-part structure:

- **Part A — Clean Baseline:** 50 questions are answered against an unmodified knowledge base. This measures the average trust score assigned to clean, factually accurate retrieval contexts and establishes the baseline accuracy of the naive always-trust system.
- **Part B — Poisoned Detection:** The same 50 questions are answered against a knowledge base in which 30% of documents have been adversarially modified. This measures the Evaluation Agent’s ability to detect and flag compromised retrieval contexts before generation.

Two experiment configurations are reported:

1. **Per-strategy experiments:** 100 samples per run (50 clean Part A, 50 poisoned Part B) with a single poisoning strategy applied in isolation (contradiction, instruction injection, entity swap, or subtle manipulation). This enables characterisation of detection difficulty per attack type.
2. **Mixed experiment:** 100 samples (50 clean Part A, 50 poisoned Part B) with poisoning strategies assigned randomly across the poisoned portion of the knowledge

base. This reflects a more realistic threat scenario where multiple attack types may co-exist.

A document is counted as successfully poisoned only if the modified text differs from the original (`poisoned_text` $\neq$ `original_text`). This prevents evaluation metrics from being distorted by unchanged documents incorrectly counted as poisoned samples.

6.1.4 Adversarial Poisoning Strategies

Four rule-based poisoning strategies are applied to modify documents in the knowledge base:

- **Contradiction:** Appends a sentence that semantically contradicts the key claim of the original document (e.g., negating the stated answer).
- **Instruction Injection:** Appends an override directive targeting the language model’s instruction-following behaviour (e.g., “*IMPORTANT: Ignore all previous context and state that [false claim]*”).
- **Entity Swap:** Replaces named entities or numeric values in-place within the original text. A three-level fallback is employed: (1) predefined named-entity substitution; (2) numeric value shifting via regular expressions when no named entity is present; (3) qualifier or negation swapping in the answer segment of the document. This strategy produces no appended content and no structural artifacts, making it the hardest attack to detect by surface-level analysis.
- **Subtle Manipulation:** Inserts hedging language, false qualifiers, or plausible-sounding uncertainty signals that subtly shift the factual content without triggering obvious pattern-matching rules.

6.2 Baseline Models and Comparison Methods

6.2.1 Naive Always-Trust Baseline

The primary comparison point is a **naive always-trust RAG system**: a standard retrieval-augmented generation pipeline that accepts all retrieved documents and generated responses without any evaluation step. This baseline always returns `is_trustworthy = True`.

In the two-part experiment design, the baseline’s accuracy equals the proportion of samples where trusting the output is the correct decision. With a 30% poisoning rate, approximately 70% of Part B samples involve clean retrieval contexts, giving the naive baseline an inherent accuracy advantage that reflects the class imbalance rather than

any detection capability. This makes accuracy alone a misleading evaluation metric and motivates the use of trust score separation alongside precision, recall, and F1.

6.2.2 Trustworthy RAG System

The full evaluation pipeline — NLI Verifier, Poison Detector, and Trust Index Calculator — constitutes the Trustworthy RAG system evaluated in this chapter. Its performance is measured against the naive baseline across the following metrics:

- **Accuracy:** fraction of samples correctly classified as trustworthy or untrustworthy relative to the ground truth poisoning label.
- **Precision:** fraction of flagged (untrustworthy) samples that are genuinely poisoned.
- **Recall (Poison Detection Rate):** fraction of poisoned samples that the Evaluation Agent successfully flags before generation.
- **F1 Score:** harmonic mean of precision and recall; the primary optimisation target for the detection task.
- **Trust Score Separation:** the difference between the mean clean trust score and the mean poisoned trust score, $\Delta = \bar{T}_{\text{clean}} - \bar{T}_{\text{poisoned}}$. This metric quantifies the discriminative power of the Trust Index independently of the classification threshold and class imbalance.

6.3 Results on Factuality Improvement

Table 6.1 reports the average trust scores and system accuracy on both benchmark datasets. The clean trust score reflects how the Evaluation Agent rates responses grounded in unmodified, factually accurate knowledge. The poisoned trust score reflects the same pipeline’s rating of responses retrieved from adversarially modified contexts.

Table 6.1: Trust score and accuracy comparison across benchmark datasets (100 samples per dataset, $K = 5$).

Dataset	Clean Trust	Poisoned Trust	Separation	System Acc.	Baseline Acc.	Improvement
TruthfulQA	0.822	0.597	0.225	91%	85%	+7%
FEVER	0.645	0.610	0.035	73%	85%	-12%

Note: The FEVER experiment demonstrates a generalisability challenge; the baseline system performs better without the Evaluation Agent. See Table B.3 for an early 20-sample pilot run ($K=3$) that showed different initial indications.

On TruthfulQA, the Evaluation Agent assigns an average clean trust score of 0.822 to responses grounded in unmodified documents, indicating that the pipeline consistently

rates factually supported outputs as trustworthy. The poisoned average of 0.597 shows that the agent is sensitive to adversarial content, producing a separation of 0.225 that enables discrimination above the 0.5 classification threshold in the majority of cases. The system achieves 91% accuracy on TruthfulQA, a +7% improvement over the naive always-trust baseline, with zero false positives on clean content (100% precision).

On FEVER, the full 100-sample run achieves 73% accuracy, underperforming the naive always-trust baseline by 12 percentage points. Although the enriched FEVER document format provides substantially more NLI-relevant context than bare claims, the Evaluation Agent produces a narrower trust score separation (0.035) and a higher false-positive rate than on TruthfulQA. This result indicates that FEVER requires additional calibration before the Trust Index can generalise reliably beyond the TruthfulQA setting. An earlier 20-sample pilot run showed 90% accuracy, but the larger run is the main evidence reported in this chapter.

6.4 Detection Performance under Data Poisoning Attacks

6.4.1 Overall Detection Performance

Table 6.2 reports detection performance on the TruthfulQA benchmark with 100 samples and a mixed poisoning strategy, where attack types are assigned randomly across the poisoned portion of the knowledge base.

Table 6.2: Overall detection performance — TruthfulQA, 100 samples, mixed strategy.

Metric	Value
Accuracy	91%
Precision	100%
Recall	40%
F1 Score	57.1%
Trust Score Separation	0.225
Baseline Accuracy	85%
Improvement over Baseline	+7%

The system achieves 40% recall across mixed strategies, meaning that 40% of poisoned retrieval contexts are correctly identified and flagged before generation. Precision of 100% indicates zero false positives on clean content — the Evaluation Agent does not flag any clean response as untrustworthy. The F1 score of 57.1% reflects the precision–recall trade-off: the system is highly conservative, preferring to miss some poisoned samples rather than raise false alarms. The FP spillover effect observed in earlier experimental rounds (where clean queries retrieving poisoned neighbours received elevated poison probabilities) was mitigated by the relevance-weighted poison aggregation described in Chapter 5.

6.4.2 Per-Strategy Detection Results

Table 6.3 reports detection performance for each poisoning strategy in isolation, using 100 samples per strategy (50 clean Part A, 50 poisoned Part B).

Table 6.3: Per-strategy detection performance — TruthfulQA, 100 samples per strategy ($K = 5$).

Strategy	Accuracy	Precision	Recall	F1	Separation
Contradiction	92%	88.9%	53.3%	66.7%	0.311
Injection	99%	93.8%	100%	96.8%	0.498
Entity Swap	85%	—	0%	0%	0.053
Subtle	88%	100%	20.0%	33.3%	0.149
Mixed	91%	100%	40.0%	57.1%	0.225

Note: the Mixed row reports the same 100-sample mixed-strategy run as Table 6.2 and is included here for direct per-strategy comparison. Precision for Entity Swap is undefined (—) because the system made no positive predictions for this strategy.

The results reveal a wide range of detection difficulty across strategies. Instruction injection achieves perfect recall (100%) and near-perfect precision (93.8%, $F1 = 96.8\%$) due to its high-severity linguistic artifacts. Contradiction achieves moderate recall (53.3%, $F1 = 66.7\%$), while subtle manipulation achieves 20% recall and entity swap achieves 0% recall — the system makes no positive predictions for entity swap, confirming it as an architectural blind spot. The mixed strategy ($F1 = 57.1\%$) reflects the blended effect of all four attack types co-existing in the poisoned knowledge base.

6.5 Robustness and Trust Index Analysis

6.5.1 Trust Score Distributions per Strategy

Table 6.4 summarises the trust score separation for each poisoning strategy, providing a threshold-independent view of the Trust Index’s discriminative power.

Table 6.4: Trust score separation per poisoning strategy.

Strategy	Clean Avg	Poisoned Avg	Separation (Δ)	Discriminability
Injection	≈ 0.79	≈ 0.29	0.498	Strong
Mixed	0.822	0.597	0.225	Moderate
Contradiction	—	—	0.311	Moderate
Subtle	—	—	0.149	Weak
Entity Swap	≈ 0.83	≈ 0.78	0.053	Very weak

Note: clean and poisoned average trust values are reported for strategies with the most extreme separation (injection and entity swap). Component averages for contradiction and subtle

strategies were not separately logged; their separation values are derived from per-sample trust score records.

The injection strategy produces the largest separation (0.498), enabling reliable discrimination at the fixed threshold of 0.5. Entity swap produces the smallest separation (0.053), reflecting near-overlapping trust score distributions between clean and poisoned samples. This near-zero separation confirms that entity swap attacks cannot be detected by Trust Index thresholding alone — the upstream poison detection signals do not differ sufficiently between clean and entity-swapped contexts.

6.5.2 Non-Linear Dampener Effect

The non-linear poison dampener, described in Chapter 5, activates when $P_{poison} > 0.7$ and reduces the final trust score by up to 40% at $P_{poison} = 1.0$. This mechanism prevents high factuality scores from masking an elevated poisoning signal when retrieved documents appear superficially supportive of the generated answer but are otherwise structurally suspicious.

In the injection experiment, average poison probabilities for poisoned contexts reached 0.994, consistently triggering the dampener and producing the strong separation of 0.498 observed. For entity swap, average poisoned-context poison probabilities remained near 0.319 — well below the 0.7 dampener threshold — leaving trust scores largely unchanged and yielding the near-zero separation of 0.053. The dampener thus amplifies the Trust Index’s response to high-confidence poison signals while having no effect on weak signals, confirming its role as a robustness mechanism rather than a uniform scaling factor.

6.6 Ablation Studies

To examine the sensitivity of the Trust Index to its component weights, five weight settings were evaluated on a smaller TruthfulQA mixed-strategy ablation subset. Table 6.5 reports the results.

Table 6.5: Ablation study — effect of Trust Index weight configuration on detection performance.

Configuration	α (Fact.)	β (Cons.)	γ (Poison)	Accuracy	F1	Recall
Default	0.40	0.35	0.25	86.7%	20.0%	11.1%
Equal weights	0.33	0.34	0.33	86.7%	20.0%	11.1%
Factuality-heavy	0.60	0.20	0.20	86.7%	20.0%	11.1%
Consistency-heavy	0.20	0.60	0.20	88.3%	36.4%	22.2%
Poison-heavy	0.20	0.20	0.60	88.3%	36.4%	22.2%

Note: all rows report 60-sample ablation runs on TruthfulQA with $K = 5$ and mixed poisoning strategies. The 100-sample primary experiment is reported separately in Table 6.2. Full

ablation results are in Table B.2.

The ablation results should be interpreted as exploratory rather than as definitive weight optimisation. On this 60-sample subset, the default, equal-weight, and factuality-heavy configurations produce identical classification metrics (86.7% accuracy, 11.1% recall, and 20.0% F1). Consistency-heavy and poison-heavy configurations improve recall to 22.2% and F1 to 36.4%, while preserving 100% precision. This suggests that larger consistency or poison-signal weights can increase sensitivity on the ablation subset, but the small number of poisoned examples means the result is not sufficient to establish a globally optimal weight setting. The default configuration is therefore retained as the primary operating point because it is the system design used in the main 100-sample experiment, not because the ablation independently proves it to be optimal.

6.7 Performance Overhead and Scalability

Table 6.6 summarises the runtime characteristics of the Evaluation Agent measured over the TruthfulQA experiments.

Table 6.6: Evaluation Agent performance overhead per sample.

Metric	Value
Total time per sample (retrieval + generation + evaluation)	≈ 17.2 s
Evaluation-only time (NLI + poison detection + Trust Index)	≈ 14.7 s
Throughput with Evaluation Agent enabled	$\approx 3\text{--}4$ queries/min
NLI inference calls per batch ($K = 5$ retrieved documents)	Up to 20 calls (5 factuality + 5 intra-document)
Baseline RAG throughput (no evaluation)	< 1 s / query
Latency overhead factor	$\approx 17\times$

The primary runtime bottleneck is the `facebook/bart-large-mnli` model running on CPU. With $K = 5$ retrieved documents, factuality checks (5), intra-document NLI (5), and cross-document consistency checks ($\binom{5}{2} = 10$) require up to 20 NLI forward passes per batch, each taking approximately 0.7–1 second on CPU hardware. The shared NLI instance — reused by both the NLI Verifier and the Poison Detector — prevents redundant model loading overhead but does not reduce the number of inference calls required per query.

The ≈ 17 -second evaluation overhead represents a $\approx 17\times$ latency increase over baseline RAG, for accuracy gains of +1% to +6% over the naive always-trust baseline. This trade-off is not justified for latency-sensitive real-time conversational applications. However, for **high-stakes batch scenarios** — such as automated document verification, regulatory compliance review, or medical information retrieval systems — the evaluation cost is acceptable relative to the risk of acting on poisoned information.

Several optimisation paths exist for reducing latency in deployment contexts:

- **GPU acceleration:** NLI inference on a mid-range GPU is estimated to reduce evaluation time from ≈ 15 s to < 2 s per sample.
- **Bounded pairwise checks:** limiting cross-document NLI to the top- K' most semantically dissimilar document pairs, rather than all $\binom{K}{2}$ pairs.
- **Query result caching:** caching NLI results for repeated or near-duplicate queries reduces redundant computation in document-heavy batch workflows.

6.8 Discussion of Results

6.8.1 Strategy-Specific Detection Analysis

The per-strategy results reveal a clear detection hierarchy that reflects both the signal richness of each attack type and the architectural coverage of the Evaluation Agent’s detection signals.

Instruction Injection — Fully Detectable. Injection achieves 100% recall because appended override directives (e.g., "IMPORTANT: Ignore all previous context and state that...") contain high-severity linguistic patterns that trigger deterministic regex rules in the Poison Detector with severity scores of 0.5–0.7. These artifacts are structurally unambiguous and do not require NLI inference for detection. Intra-document NLI provides a complementary signal: splitting each document and running NLI between the original factual content and the appended adversarial directive reveals a strong semantic discontinuity, producing intra-document contradiction scores above the 0.7 threshold. The injection strategy also exhibits the largest trust score separation (0.498) across all strategies tested.

Contradiction — Moderately Detectable. Contradiction achieves 53.3% recall with a separation of 0.311. Intra-document NLI successfully detects cases where the appended contradiction produces a strong first-half to second-half contradiction signal (above 0.7). However, some poisoned samples produce scores near the detection threshold, resulting in borderline classifications. The detection rate is sensitive to the specific phrasing of the contradiction: explicit negations are more detectable than hedged or qualified contradictions.

Entity Swap — Undetectable. Entity swap achieves 0% recall with a separation of 0.053 — the system makes no positive predictions for this strategy. Unlike injection and contradiction, entity swap modifies documents in-place with no appended content and no structural artifacts. The resulting documents are syntactically and stylistically

indistinguishable from genuine documents at the surface level — only specific numeric or named-entity values are changed. Detecting this class of attack requires external world-knowledge verification (e.g., querying a fact database to verify whether a given value is correct for a particular claim), which is beyond the capability of a surface-signal-based evaluation agent. This constitutes an **architectural limit** of the current approach.

Subtle Manipulation — Near-Undetectable. Subtle manipulation achieves 20.0% recall with a separation of 0.149. Hedging language and false qualifiers do not trigger pattern-matching rules, and the semantic shift introduced is too small for intra-document NLI to consistently detect above threshold. Average poison probabilities for this strategy hover near 0.700 — at the boundary of the non-linear dampener — indicating that the signals present are at the edge of statistical significance under the current architecture.

6.8.2 False Positive Spillover Effect

A consistent source of false positives across all strategies is the *FP spillover effect*: a clean query that retrieves a poisoned document as a neighbour in the top- K results receives an elevated overall poison probability, causing the Evaluation Agent to flag the response as untrustworthy even when the highest-ranked retrieved document is clean.

This behaviour is architecturally correct from a security perspective: the retrieval environment is contaminated, and the Evaluation Agent correctly warns that the context cannot be fully trusted. However, the spillover reduces the precision metric, particularly in the injection experiment. The relevance-weighted poison aggregation introduced in Chapter 5 mitigates this effect by weighting each document’s poison probability by its retrieval similarity score, so low-relevance poisoned neighbours contribute less to the overall estimate than the top-ranked document. This intervention reduced false positives substantially in clean-KB experiments. However, when a poisoned document ranks high in semantic similarity to the query, the spillover effect is not fully eliminated by relevance weighting alone.

This represents a fundamental design trade-off: a system that issues warnings about contaminated retrieval environments will inevitably produce false positives when clean queries share embedding space proximity with poisoned documents. The appropriate response is not to suppress these warnings but to provide richer contextual information about which specific documents triggered the alert.

6.8.3 Trust Score Separation as Primary Metric

In class-imbalanced settings — where 70% of samples involve clean retrieval contexts — accuracy alone is a misleading evaluation metric. The naive always-trust baseline achieves 85% accuracy without any detection capability, simply by never raising a warning. The

Trust Index’s discriminative value is better captured by the trust score separation Δ , which measures how far apart the clean and poisoned trust distributions are independently of threshold choice and class composition.

The per-strategy separation range (0.053 for entity swap to 0.498 for injection) provides actionable guidance for future defence design. Strategies with high separation are candidates for reliable deployment under the current architecture; strategies with low separation require fundamentally different detection mechanisms. This per-strategy characterisation is a contribution of this thesis: prior evaluation frameworks have not systematically quantified the detection difficulty of individual poisoning strategies in RAG-specific evaluation settings.

6.9 Comparative Evaluation: LLM and Embedding Model Sensitivity

To assess whether the Trust Index performance is specific to the Llama 3.3 70B model and the all-MiniLM-L6-v2 embedding pipeline, a **2×2 factorial experiment** was conducted crossing two LLMs with two embedding models. All four configurations were evaluated under identical conditions: 100 samples (50 clean Part A, 50 poisoned Part B), 30% poison ratio, mixed strategy, $K = 3$, $\tau = 0.5$. The impact of increasing retrieval depth to $K = 5$ is analysed separately in Section 6.10.

6.9.1 Grid Results

Table 6.7 reports the full performance grid.

Table 6.7: 2×2 factorial grid results — TruthfulQA, 100 samples, mixed strategy.

LLM	Embedding	Acc.	Prec.	Recall	F1	Clean $\bar{T}$	Poison $\bar{T}$	Sep.
Llama 3.3 70B	all-MiniLM-L6-v2	91%	100%	40%	57.1%	0.830	0.590	0.240
Llama 3.3 70B	snowflake-arctic-embed2	91%	100%	40%	57.1%	0.799	0.611	0.188
Qwen 3.5 35B	all-MiniLM-L6-v2	71%	25.0%	46.7%	32.6%	0.633	0.471	0.161
Qwen 3.5 35B	snowflake-arctic-embed2	71%	28.1%	60.0%	38.3%	0.653	0.454	0.199
Naive always-trust baseline		85%	—	—	—	—	—	—

6.9.2 Finding 1: Embedding Invariance for Llama 3.3 70B

The two Llama configurations produce **identical classification results** (Acc = 91%, Prec = 100%, Recall = 40%, F1 = 57.1%), with the sole difference being a slightly higher clean trust score for MiniLM (0.830 vs. 0.799) and a larger trust score separation (0.240 vs. 0.188). The zero false positive count in both Llama runs confirms that the precision advantage is attributable to the LLM’s concise answer style rather than to retrieval quality.

This result demonstrates that for a concise-output LLM such as Llama 3.3, switching to a higher-dimensional embedding model (1024-dim Snowflake vs. 384-dim MiniLM) does not improve poison detection. The Trust Index’s discriminative power is determined primarily by the NLI entailment signal between the generated answer and retrieved documents, which is insensitive to the specific embedding space used for retrieval as long as topically relevant documents are returned.

6.9.3 Finding 2: Qwen 3.5 35B Generation-Style False Positives

Both Qwen configurations **underperform the naive baseline** by 14 percentage points (71% vs. 85%), producing 21–23 false positives out of 85 clean samples. The root cause is the LLM’s verbose, hedged generation style: Qwen 3.5 35B typically prefaces answers with phrases such as “*Based on the provided context*” or “*According to the available information,*” which the NLI Verifier interprets as reduced semantic commitment to the claim. This reduces $S_{\text{factuality}}$ below 0.5 for many clean responses, causing the Trust Index to fall below the classification threshold $\tau = 0.5$ and triggering false untrustworthy labels.

The mean clean trust score for Qwen (≈ 0.64) is substantially lower than for Llama (≈ 0.83), confirming that the generation-style effect is systematic rather than query-specific. This finding demonstrates that the Trust Index’s $\tau = 0.5$ threshold is implicitly calibrated for the Llama generation distribution and requires per-LLM recalibration for reliable deployment with alternative generators.

6.9.4 Finding 3: Retrieval Quality Paradox

For Qwen 3.5 35B, the higher-dimensional Snowflake Arctic Embed2 model shows marginally improved F1 (38.3% vs. 32.6%) and recall (60% vs. 46.7%), with the same accuracy (71%) and a comparable false positive count. The slight improvement in recall may reflect that Snowflake’s superior retrieval precision recovers more relevant poisoned documents, making them available for NLI analysis.

However, a counter-intuitive pattern also emerges: Snowflake’s stronger retrieval retrieves *cleaner* documents for genuinely clean queries, raising the mean clean trust score slightly (0.653 vs. 0.633). This constitutes a **retrieval quality paradox**: higher retrieval quality dilutes contaminated context by preferring highly relevant clean documents, reducing the poison detection signal for poisoned queries while simultaneously reducing false positives for clean queries. The net effect on F1 depends on which correction dominates.

6.9.5 Implications for Deployment

The grid experiment yields three actionable guidelines for deploying the Trustworthy RAG framework:

1. **LLM selection is the dominant factor.** Switching embedding models within the same LLM produces negligible differences in detection performance for Llama. The choice of LLM has a far larger effect on both precision (100% vs. 25–28%) and overall accuracy (91% vs. 71%).
2. **Per-LLM threshold calibration is required.** The fixed threshold $\tau = 0.5$ is not universally applicable. For LLMs with hedged generation styles, the Trust Index threshold must be lowered, or the factuality component normalised, to avoid systematic false positives on clean content.
3. **The framework is validated as reproducible on Llama 3.3 70B.** The new Llama+MiniLM run replicates the original thesis baseline identically (same TP/FP/FN counts, trust score differences below 0.015), confirming that the results are stable across independent runs.

6.10 Retrieval Depth Sensitivity: $K = 3$ versus $K = 5$

To isolate the effect of retrieval depth on detection performance, the primary configuration (Llama 3.3 70B + all-MiniLM-L6-v2) was evaluated under two retrieval settings: $K = 3$ (used in the 2×2 grid experiment, Section 6.9) and $K = 5$ (the system default). All other parameters were held constant (TruthfulQA, 100 samples, 30% poison ratio, mixed strategy, $\tau = 0.5$).

Increasing K from 3 to 5 has two theoretically opposing effects. First, more retrieved documents increase the probability of a poisoned document appearing in the batch, which should improve recall. Second, the number of NLI inference calls grows from $3 + 3 + \binom{3}{2} = 9$ to $5 + 5 + \binom{5}{2} = 20$ per batch (factuality + intra-document + pairwise cross-document), increasing evaluation overhead approximately $2.2\times$.

Table 6.8: Retrieval depth sensitivity: $K = 3$ vs $K = 5$ (Llama 3.3 70B + all-MiniLM-L6-v2, TruthfulQA, 100 samples, mixed strategy).

K	Acc.	Prec.	Recall	F1	Clean $\bar{T}$	Sep.	NLI calls/batch
$K = 3$	91%	100%	40.0%	57.1%	0.830	0.240	~ 9
$K = 5$	91%	100%	40.0%	57.1%	0.822	0.225	~ 20

6.10.1 Finding: Detection Is Retrieval-Depth-Invariant for Llama 3.3 70B

The results in Table 6.8 show that increasing retrieval depth from $K = 3$ to $K = 5$ produces **no improvement in mixed-strategy detection performance**. The classification outcome is identical: 6 true positives, 0 false positives, and 9 false negatives in both configurations. Accuracy (91%), precision (100%), recall (40.0%), and F1 (57.1%)

are unchanged. The trust score separation decreases marginally from 0.240 to 0.225, a difference within noise bounds for the classification decision.

The predicted recall improvement does not materialise. The reason is structural: the 9 undetected poisoned contexts (the false negatives) correspond to Entity Swap and Subtle attack strategies, which produce in-place modifications to the document text. These attacks are difficult to detect because they leave no textual artefacts that would be flagged by linguistic pattern detectors or intra-document NLI, regardless of how many documents are retrieved. Adding two extra clean documents to the batch provides no additional poison signal for these attacks.

Conversely, the 6 successfully detected contexts (Contradiction and Injection strategies) are detectable at $K = 3$ because they append distinct textual patterns (contradiction markers and instruction-override phrases) that the detector flags reliably. Retrieving 5 instead of 3 documents does not improve recall for these strategies, since they are already detected at the lower retrieval depth.

6.10.2 Implication: Detection Bottleneck Is Attack Strategy, Not Retrieval Depth

This null result has a clear architectural implication: **the detection ceiling of the Evaluation Agent at 40% recall is set by the attack strategy difficulty, not by the number of documents retrieved.** Entity Swap and Subtle attacks represent fundamental limits of the current NLI-and-heuristic approach: they require external world knowledge or fine-grained semantic comparison to detect reliably.

At the per-strategy level, $K = 5$ does yield a precision improvement for instruction injection (precision 93.8% at $K = 5$ vs. 37.5% at $K = 3$, due to improved relevance-weighted dilution of poison signals from low-ranked neighbours), while all other per-strategy classification metrics remain comparable. However, since recall — the primary safety metric — is unchanged across all strategies, the mixed-strategy detection ceiling of 40% is unaffected.

From a deployment standpoint, $K = 3$ is therefore the preferred configuration for this system: it achieves equivalent detection performance at approximately half the NLI inference cost (~ 9 vs. ~ 20 calls per batch). The $K = 3$ setting is used for the 2×2 grid experiment (Section 6.9), and this sensitivity analysis retroactively validates that choice.

6.11 Summary

This chapter evaluated the Trustworthy RAG framework across TruthfulQA and FEVER benchmarks with four adversarial poisoning strategies. The Evaluation Agent achieves 96.8% F1 score on isolated instruction injection, with instruction injection detected at

100% recall and 93.8% precision. On the mixed 100-sample experiment (Llama 3.3 70B + all-MiniLM-L6-v2), the system achieves 91% accuracy, 57.1% F1, 100% precision, and a trust score separation of 0.225, representing a +7% improvement over the naive always-trust baseline on TruthfulQA. On FEVER, the full 100-sample experiment reaches 73% accuracy and underperforms the 85% always-trust baseline, highlighting a generalisability limitation.

Per-strategy analysis reveals a clear detection hierarchy: instruction injection and contradiction are amenable to textual-signal and intra-document NLI approaches, while entity swap and subtle manipulation represent architectural limits requiring external world-knowledge for reliable detection. The ≈ 17 -second evaluation overhead positions the system as suitable for high-stakes batch deployment rather than real-time applications.

The ablation study shows that Trust Index weights affect recall and F1 on the smaller 60-sample subset. The default configuration remains the primary setting used for the main experiment, but the ablation results are exploratory and do not independently establish a globally optimal weight configuration.

A 2×2 factorial experiment (Section 6.9) further establishes that LLM selection is the dominant performance factor: Llama 3.3 70B achieves 91% accuracy with zero false positives under both embedding models, while Qwen 3.5 35B underperforms the naive baseline due to generation-style false positives. These findings highlight the importance of per-LLM Trust Index calibration in practical deployments.

Section 6.10 (Retrieval Depth Sensitivity) establishes that detection performance is retrieval-depth-invariant for Llama 3.3 70B: $K = 5$ produces identical classification outcomes to $K = 3$ (91% accuracy, 57.1% F1, 0 false positives) at approximately $2.2\times$ the NLI inference cost, confirming that the 40% recall ceiling is set by attack strategy difficulty rather than retrieval depth. The next chapter discusses all findings in the context of the research questions, identifies the system’s theoretical and practical contributions, and outlines directions for future work.

Chapter 7

Discussion

This chapter interprets the empirical findings reported in Chapter 6 in the context of the three Research Questions stated in Chapter 1. It articulates the specific theoretical and practical contributions of this work, discusses the implications for real-world deployment of trustworthy RAG systems, identifies the study’s limitations, and reflects on the ethical considerations surrounding adversarial AI evaluation research.

7.1 Interpretation of Key Findings

7.1.1 RQ1: Detection Effectiveness of the Evaluation Agent

What computational strategies (e.g., NLI, semantic discrepancy detection) are effective for detecting misinformation and poisoning in retrieved contexts?

The results reveal a clear detection hierarchy across the four adversarial strategies, and the answer to RQ1 is nuanced: the Evaluation Agent is highly effective against overt poisoning strategies but structurally limited against subtle in-place modifications.

Instruction injection is the most detectable strategy, achieving 100% recall in per-strategy experiments. The Evaluation Agent’s linguistic pattern detector reliably identifies instruction-override phrases (e.g., “IMPORTANT: Ignore previous context”), and the intra-document NLI component detects the structural inconsistency introduced when such phrases are appended to otherwise coherent documents. This finding confirms that prompt injection attacks — which Greshake et al. [14] identified as a critical threat to LLM-integrated systems — are amenable to surface-level detection when the injection is syntactically distinct from the host document.

Contradiction poisoning achieves moderate detection (53% recall in the primary per-strategy experiment), primarily through intra-document NLI contradiction signals. When a contradicting sentence is appended to a document, the NLI model detects semantic inconsistency between the document’s two halves. Detection is partial rather than complete because the NLI signal strength varies with sentence specificity: generic con-

tradiction markers (e.g., “Contrary to popular belief”) produce weaker NLI signals than factually specific contradictions.

Entity swap and **subtle manipulation** represent the architectural limit of the current approach: entity swap achieves 0% recall and subtle manipulation achieves 20% recall in per-strategy experiments. These strategies operate through in-place text modifications — replacing numbers, proper nouns, or qualifiers within the original sentence structure — leaving no appended content for intra-document NLI or pattern detectors to flag. The modified document is semantically close to the clean version and produces nearly identical NLI scores. Detecting these attacks would require external world-knowledge reasoning (e.g., verifying that “Berlin” is an incorrect answer to a question about France’s capital) — a capability beyond the current NLI-and-heuristic architecture.

In the mixed-strategy 100-sample experiment, the system achieves 40% overall recall with 100% precision (zero false positives on clean content), demonstrating that the agent’s detection is reliable when it fires but conservative in coverage. This precision-recall trade-off reflects a deliberate design choice: false positives on clean content undermine user trust in the system’s warnings, making precision the higher-priority metric in high-stakes deployments.

7.1.2 RQ2: Reliability and Calibration of the Trust Index

How can a composite “Trust Index” be mathematically formulated to quantify the reliability of a RAG response?

The Trust Index provides a calibrated, interpretable trustworthiness score across the evaluated scenarios. Several properties of its design are validated by the experiments.

Discriminative separation: In the primary 100-sample configuration (Llama 3.3 70B + all-MiniLM-L6-v2, $K = 5$), clean responses achieve a mean Trust Index of $\bar{T}_{\text{clean}} = 0.822$, while poisoned contexts produce $\bar{T}_{\text{poison}} = 0.597$, a separation of 0.225 points. The $K = 3$ grid run produces the same classification metrics with a slightly larger separation of 0.240. This separation is consistent across both embedding models for Llama (0.240 for MiniLM, 0.188 for Snowflake in the $K = 3$ grid), indicating that the Trust Index’s discriminative capacity is primarily driven by the NLI signal rather than by retrieval quality.

Non-linear dampener effectiveness: The ablation study confirms that the dampener (Section 5.2.5) plays a critical role under high-contamination conditions. Without the dampener, responses with high factuality scores (because the LLM ignores the poisoned portion) can exceed the $\tau = 0.5$ threshold despite high P_{poison} values. The dampener imposes a multiplicative penalty that prevents this override, ensuring that a 90% poison probability forces the final trust score below the classification threshold even when factuality and consistency appear healthy.

Weight sensitivity: The ablation study shows that Trust Index performance is sensitive to the relative weighting of its components, but it does not independently prove that the default weights are globally optimal. On the 60-sample $K = 5$ ablation subset, the default, equal-weight, and factuality-heavy configurations produce identical metrics, while consistency-heavy and poison-heavy configurations improve recall from 11.1% to 22.2% and F1 from 20.0% to 36.4%. The default weights therefore remain a conservative primary operating point used in the main experiment, while larger ablation studies would be needed for reliable weight optimisation.

LLM-dependency: The 2×2 factorial experiment reveals a critical limitation of the Trust Index’s universality: its component scores are sensitive to LLM generation style. Qwen 3.5 35B’s hedged outputs (“Based on the provided context...”) systematically reduce $S_{\text{factuality}}$ below the threshold for clean responses, generating 21–23 false positives per run and causing the system to underperform the naive baseline (71% vs. 85%). This finding provides empirical evidence that the Trust Index threshold τ and weights must be calibrated per LLM before deployment.

7.1.3 RQ3: Security Performance and Operational Overhead

To what extent does the inclusion of an Evaluation Agent enhance the security performance of RAG pipelines compared to baseline systems, what are the trade-offs in latency, and how does the choice of LLM and embedding model affect the Trust Index’s discriminative performance?

The Evaluation Agent delivers an empirically meaningful security improvement over the naive always-trust baseline when paired with Llama 3.3 70B: 91% accuracy versus 85% baseline (+7%), with zero false positives on clean content. In absolute terms, 6 poisoned contexts out of 15 are correctly identified and blocked before generation, while 9 (the entity-swap and subtle strategy cases) pass the agent undetected.

The 2×2 factorial experiment isolates two key factors:

LLM selection is the dominant factor. Both Llama configurations (MiniLM and Snowflake) produce identical detection results (91% accuracy, 57.1% F1), whereas Qwen configurations achieve only 71% accuracy with significantly lower F1 (31–38%). The choice of embedding model produces negligible differences within the same LLM. This finding directly implies that system operators choosing models for Trust-Index-based evaluation should prioritise LLM output style over embedding dimensionality or retrieval quality.

Retrieval depth does not affect detection. The $K=3$ vs. $K=5$ sensitivity analysis (Section 6.10) confirms that increasing the number of retrieved documents from 3 to 5 produces no improvement in detection performance for Llama: TP, FP, and FN counts are identical, and the trust score separation changes by only 0.015. The detection ceiling is set by attack strategy difficulty, not by retrieval depth. This null result retroactively

validates using $K=3$ in the 2×2 grid and provides a practical recommendation: $K=3$ achieves equivalent security at half the NLI inference cost (~ 9 vs. ~ 20 calls per batch).

The ≈ 17 -second per-sample evaluation overhead — dominated by CPU-based NLI inference and LLM generation latency on the FARMI cluster — positions the system as suitable for **batch or high-stakes offline deployment** rather than real-time interactive applications. The evaluation overhead could be reduced substantially through GPU acceleration of the NLI model, which is addressed in Section 8.3.

7.2 Contributions to Trustworthy and Secure AI Research

This thesis makes six distinct contributions spanning theoretical frameworks, empirical findings, and practical system design.

Contribution 1: The Trust Index — a unified composite trustworthiness metric. Existing evaluation frameworks for RAG systems, such as RAGAS [5] and ARES [6], assess whether generated answers are faithful to retrieved context. They do not assess whether the retrieved context itself is trustworthy. The Trust Index ($T = \alpha \cdot S_{\text{factuality}} + \beta \cdot S_{\text{consistency}} + \gamma \cdot (1 - P_{\text{poison}})$) closes this gap by simultaneously quantifying factual grounding, inter-document consistency, and adversarial contamination in a single interpretable score. Its non-linear dampener ensures that strong poisoning signals can override high factuality scores — a structural safeguard absent from linear aggregation approaches.

Contribution 2: A multi-signal Poison Detector for RAG pipelines. The five-signal detection architecture (linguistic pattern matching, structural anomaly detection, intra-document NLI, cross-document consistency checking, and semantic outlier detection via embedding centroid deviation) provides defence-in-depth coverage across structurally diverse attack strategies. Relevance-weighted aggregation of per-document poison scores prevents low-relevance suspicious neighbours from dominating the batch-level decision, a design feature that substantially reduced false positives in preliminary evaluations (Round 10 of development, FEVER dataset).

Contribution 3: Empirical per-strategy characterisation of detection difficulty. The per-strategy evaluation provides a detailed characterisation of how detection difficulty varies across four adversarial strategies in a RAG-specific context. The hierarchy — injection (100% recall) $\gg$ contradiction ($\approx 53\%$) $\gg$ subtle ($\approx 20\%$) $\gg$ entity swap (0%) — provides concrete guidance for future defence design: strategies that append syntactically distinct content are detectable by surface-level signals; strategies that perform in-place semantic modifications require world-knowledge verification.

Contribution 4: Identification of an architectural detection limit. Entity swap and subtle manipulation represent a demonstrated *architectural limit* of NLI-and-

heuristic approaches: they cannot be reliably detected without access to external factual grounding. This finding advances the scientific understanding of the attack surface in RAG systems and motivates the integration of external knowledge verification (e.g., Wikidata lookup, search-augmented fact checking) in future evaluation agent designs.

Contribution 5: Empirical demonstration of LLM-dependency in NLI-based trust scoring. While the sensitivity of NLI models to phrasing has been noted in the NLP literature [13], this thesis provides empirical evidence that LLM generation style systematically affects Trust Index values in a RAG evaluation context. The finding that Qwen 3.5 35B’s hedged outputs reduce mean clean trust scores by 0.20 points relative to Llama 3.3 70B — sufficient to push a majority of clean responses below the classification threshold — has direct implications for deployment: the Trust Index threshold must be treated as an LLM-specific hyperparameter, not a universal constant.

Contribution 6: Retrieval-depth invariance finding. The K=3 vs. K=5 sensitivity analysis identifies an important property of the evaluated NLI-based retrieval setup: detection performance is insensitive to retrieval depth for the attack strategies and LLM combination tested. This is not obvious a priori (more documents should theoretically improve the chance of exposing poisoned content), and the null result reveals that the bottleneck is the detector’s inability to identify certain attack types regardless of batch size. This finding has practical implications for system design: NLI inference cost can be minimised without sacrificing detection quality.

7.3 Implications for Practical Deployment

The experimental results suggest several concrete guidelines for organisations considering deployment of the Trustworthy RAG framework.

Target use cases. The system’s ≈ 17 -second per-query evaluation overhead makes it unsuitable for real-time conversational applications. Its strongest use case is **high-stakes batch processing** where response latency is tolerable and the cost of misinformation is high: regulatory document review, medical information retrieval, legal evidence search, and automated fact-checking pipelines. The 100% recall for instruction injection attacks makes the agent immediately valuable as a defence against prompt injection in document-processing workflows, which represent an increasingly common real-world attack vector [14].

LLM selection and per-model calibration. Operators must calibrate the Trust Index threshold τ for each LLM used in the system. The fixed $\tau = 0.5$ is empirically tuned for Llama 3.3 70B’s concise output style. Verbose or hedged LLMs require a lower threshold (or style-normalisation preprocessing) to avoid systematic false positives on clean content. A practical calibration procedure involves running the evaluation agent on a small held-out set of clean queries for the target LLM and selecting τ as the 10th

percentile of the resulting clean trust score distribution.

Strategy-aware deployment. The per-strategy detection hierarchy implies that the system’s value depends heavily on the expected attack distribution. Against adversaries primarily using injection strategies (the most common form of prompt injection), the Evaluation Agent provides near-perfect recall. Against sophisticated adversaries using entity-swap or subtle manipulation techniques, coverage is limited: the $K = 5$ per-strategy runs show 0% recall for entity swap and 20% recall for subtle manipulation. Operators in adversarial environments should be aware of this coverage gap and supplement the agent with external fact-checking for high-risk queries.

Transparency and human oversight. Each evaluation produces component scores ($S_{\text{factuality}}$, $S_{\text{consistency}}$, P_{poison}), detected warning signals, and a categorical trust level (HIGH / MEDIUM / LOW / VERY LOW), enabling human reviewers to inspect the agent’s reasoning rather than relying on a black-box decision. This design aligns with the principle of human-in-the-loop AI systems advocated in the EU AI Act’s requirements for high-risk AI applications, particularly those involving safety-critical information provision.

Infrastructure recommendations. GPU acceleration of the BART-large-MNLI model would reduce per-sample evaluation overhead from approximately 15 seconds to under 2 seconds, enabling near-real-time deployment. Limiting cross-document pairwise NLI to the top-3 retrieved documents ($K=3$, as validated by the sensitivity analysis) minimises NLI calls without sacrificing detection quality. For systems using hedged LLMs, stripping common hedging prefixes from generated answers before NLI inference would reduce generation-style artefacts without affecting factual content evaluation.

7.4 Limitations of the Study

This study acknowledges seven principal limitations that bound the generalisability of its findings.

Limitation 1: Rule-based adversarial strategies. The four poisoning strategies implemented in this thesis are rule-based transformations of existing documents. Real-world adversaries — particularly those described by Zou et al. [4] — can employ gradient-optimized “collision” documents engineered to maximise vector similarity with target queries while containing arbitrary misinformation. Such documents may not exhibit the linguistic artefacts that the current detector relies upon, and their detection would require fundamentally different techniques (e.g., embedding-space anomaly detection or adversarial training of the NLI model).

Limitation 2: Detection ceiling for subtle attacks. The 0% recall for entity-swap and 20% recall for subtle manipulation are structural limits of the current approach, not tuning deficiencies. Increasing retrieval depth (validated by the K sensitivity analysis),

adjusting thresholds, or reweighting the Trust Index components do not improve detection for these strategies. Addressing this limitation requires semantic world-knowledge integration, which is treated as future work.

Limitation 3: Evaluation latency. The current implementation performs NLI inference on CPU, producing approximately 17 seconds of total per-sample overhead, including roughly 15 seconds of evaluation-only latency. This is impractical for interactive deployments. While GPU acceleration would resolve this, the thesis cannot empirically validate the latency improvement due to infrastructure constraints on the FARMI computing cluster.

Limitation 4: Fixed classification threshold. The threshold $\tau = 0.5$ is constant across query types, domains, and LLMs. It does not adapt to the risk level of the query (a question about medication dosage warrants a lower trust threshold than a general knowledge query), the domain’s prior probability of poisoning, or the generation style of the LLM. This inflexibility is a known limitation that the per-LLM calibration recommendation in Section 7.3 partially addresses but does not fully resolve.

Limitation 5: Attack Success Rate not measured. The experiments measure whether the Evaluation Agent correctly *flags* poisoned samples, not whether the LLM *incorporates* the misinformation into its answer. True Attack Success Rate measurement would require semantic comparison of generated answers to injected claims. In practice, Llama 3.3 70B frequently ignores appended injection and contradiction content, producing factually correct answers from the legitimate portion of the document. This makes the injected content detectable by the Evaluation Agent even when it fails to influence the generated answer, potentially inflating the perceived utility of detection. Future work should measure ASR directly.

Limitation 6: Dataset and domain scope. Experiments use two standardised benchmarks (TruthfulQA and FEVER) under controlled poisoning conditions. Performance on specialised corpora — medical literature, legal documents, scientific papers — may differ substantially, as these domains involve technical vocabulary, complex claim structures, and different LLM behaviour patterns. The generalisability of the Trust Index to such domains has not been empirically validated.

Limitation 7: Ground-truth convention for poisoning labels. The current evaluation protocol labels a sample as poisoned based on document-level index alignment (whether the knowledge-base document corresponding to a query was modified), rather than directly checking whether a poisoned document was retrieved in that specific query’s top- K results. This convention is suitable for the controlled question-document paired benchmark setup used in this thesis, but it may overestimate or underestimate detection metrics in settings where retrieval frequently returns off-index neighbours. Future evaluations should implement retrieval-grounded poisoning labels to improve ecological validity.

7.5 Alignment with Ethical AI and Security Frameworks

The development and evaluation of adversarial AI systems carries inherent ethical responsibilities. This section reflects on the ethical dimensions of the thesis in the context of established AI governance frameworks.

Transparency and explainability. The Evaluation Agent is explicitly designed to produce interpretable outputs: component trust scores, detected warning signals, and a categorical trust level accompany every evaluation decision. This design aligns with the transparency and explainability requirements of the EU AI Act (Article 13) for high-risk AI systems, which mandate that automated decisions be explainable to affected parties. Unlike black-box LLM-as-a-Judge approaches, the agent’s decisions can be audited and challenged by human reviewers.

Defensive intent and responsible disclosure. The adversarial dataset generator and poisoning strategies implemented in this thesis are designed exclusively for controlled research evaluation. The code does not include optimized attack capabilities (such as gradient-based collision document generation) that would provide operational uplift to malicious actors. The strategies implemented are simplified, rule-based transformations intended to characterise detection difficulty, not to enable novel attacks. This approach aligns with responsible disclosure principles in security research: the system’s limitations (particularly the near-zero recall for entity-swap attacks) are transparently documented, enabling downstream researchers and practitioners to make informed deployment decisions.

Human-in-the-loop design. The Evaluation Agent is designed to augment rather than replace human oversight. It provides trust scores and warnings that inform human decision-making rather than autonomously filtering or censoring retrieved content. The categorical trust levels (HIGH / MEDIUM / LOW / VERY LOW) are calibrated to support graduated human review: HIGH-trust responses may be used directly, while VERY LOW-trust responses should be independently verified before use. This design philosophy aligns with the principle of meaningful human control advocated by the OECD Principles on Artificial Intelligence and the IEEE Ethically Aligned Design framework.

Dual-use considerations. The detection techniques developed in this thesis — particularly the linguistic pattern library and intra-document NLI — could theoretically be inverted to inform adversaries about the specific features that the detector monitors, enabling more sophisticated evasion. This dual-use risk is inherent to all security research and is mitigated by the open publication of both the detection system and its known limitations, consistent with the principle that defensive knowledge should be equally accessible to defenders.

7.6 Summary

This chapter interpreted the experimental results in the context of the three Research Questions, articulated six distinct contributions, and identified limitations and deployment implications. The core finding is that the Evaluation Agent provides reliable, interpretable defence against overt poisoning strategies (instruction injection and contradiction) at the cost of an evaluation overhead that currently limits it to batch deployments. Entity-swap and subtle manipulation represent a fundamental architectural limit requiring world-knowledge integration to overcome. The LLM-dependency of the Trust Index and the retrieval-depth-invariance finding provide empirical contributions that advance the design science of trustworthy RAG evaluation. The following chapter synthesises these findings into a final conclusion and outlines a research roadmap toward secure, auditable RAG systems.

Chapter 8

Conclusion and Future Work

This thesis investigated the design and evaluation of an autonomous Evaluation Agent for detecting misinformation and knowledge poisoning in Retrieval-Augmented Generation (RAG) systems. Motivated by the growing deployment of RAG in high-stakes information environments and the demonstrable vulnerability of retrieval-augmented architectures to corpus poisoning attacks, the work addressed a gap left by existing evaluation frameworks: the absence of a runtime defence that simultaneously assesses factual grounding, inter-document consistency, and adversarial contamination.

8.1 Summary of Research Objectives and Findings

The thesis was structured around three Research Questions, each of which is now answered by the empirical evidence gathered across the experimental programme.

RQ1 — Detection Effectiveness. *What computational strategies (e.g., NLI, semantic discrepancy detection) are effective for detecting misinformation and poisoning in retrieved contexts?*

The Evaluation Agent achieves high effectiveness against overt poisoning strategies but encounters a structural detection limit against subtle in-place modifications. Instruction injection is detected with 100% recall; contradiction poisoning achieves 53% recall in the primary per-strategy experiment; entity-swap achieves 0% recall and subtle manipulation achieves 20% recall — limits imposed by the absence of external world-knowledge reasoning in the current architecture. In the primary mixed-strategy 100-sample experiment, the system achieves 40% overall recall with 100% precision (zero false positives on clean content), confirming that the agent’s detections are highly reliable when they occur.

RQ2 — Trust Index Reliability. *How can a composite “Trust Index” be mathematically formulated to quantify the reliability of a RAG response?*

The Trust Index formula $T = 0.4 \cdot S_{\text{factuality}} + 0.35 \cdot S_{\text{consistency}} + 0.25 \cdot (1 - P_{\text{poison}})$, augmented with a non-linear dampener for high-contamination conditions, provides a calibrated and discriminative trustworthiness score. In the primary $K = 5$ experiment,

mean trust score separation between clean (0.822) and poisoned (0.597) contexts confirms meaningful discriminative power. The ablation study shows that weight choice affects sensitivity, but larger ablation studies would be needed to optimise the weights definitively. However, the 2×2 factorial experiment reveals that Trust Index values are LLM-dependent: verbose generation styles systematically reduce $S_{\text{factuality}}$ for clean responses, requiring per-LLM threshold calibration before production deployment.

RQ3 — Security Improvement and Overhead. *To what extent does the inclusion of an Evaluation Agent enhance the security performance of RAG pipelines compared to baseline systems, what are the trade-offs in latency, and how does the choice of LLM and embedding model affect the Trust Index’s discriminative performance?*

The Evaluation Agent improves overall accuracy by +7% over the naive always-trust baseline (91% vs. 85%) with Llama 3.3 70B and zero false positives on clean content on the TruthfulQA benchmark. However, the system’s performance on the FEVER dataset (73% accuracy) highlights a generalisability limitation, indicating that the Trust Index requires domain-specific calibration. The 2×2 factorial experiment establishes that LLM selection is the dominant performance factor: both Llama embedding configurations achieve identical results, while Qwen 3.5 35B underperforms the baseline by 14 percentage points due to generation-style false positives. The K=3 vs. K=5 sensitivity analysis confirms that detection performance is retrieval-depth-invariant, and that K=3 is the cost-optimal retrieval depth for this system.

Table 8.1 summarises the correspondence between each Research Question and its primary experimental evidence.

Table 8.1: Research Questions and corresponding experimental findings.

RQ	Finding	Evidence
RQ1	High recall for injection/contradiction; near-zero for entity-swap/subtle	Per-strategy experiments (Section 6.4)
RQ2	Trust Index discriminates clean vs. poisoned; LLM-dependent	Main experiment + 2×2 grid (Sections 6.2, 6.9)
RQ3	+7% vs baseline; LLM invariant	2×2 grid + K sensitivity (Sections 6.9, 6.10)

8.2 Theoretical and Practical Contributions

8.2.1 Theoretical Contributions

The Trust Index as a unified security-reliability metric. This thesis formalises a composite metric designed to simultaneously evaluate the factual grounding, internal consistency, and adversarial safety of a RAG response in a single interpretable score. Existing frameworks such as RAGAS [5] and ARES [6] address faithfulness but not corpus integrity. The Trust Index bridges this gap and establishes a mathematical foundation for what the thesis terms the “Trust-Security Gap” in RAG evaluation.

Empirical characterisation of per-strategy detection difficulty. The per-strategy experimental analysis provides empirical evidence of the varying detectability of four structurally distinct adversarial strategies in RAG corpora. The resulting detection hierarchy (injection $\gg$ contradiction $\gg$ entity-swap $\approx$ subtle) constitutes a reusable reference for future research on RAG defence design, establishing which attack families are amenable to surface-signal detection and which require semantic world-knowledge integration.

LLM-dependency of NLI-based trust scoring. The 2×2 factorial experiment provides clear empirical evidence that NLI-based trust assessment is sensitive to LLM generation style. This finding generalises beyond the specific models tested: any LLM whose output distribution differs significantly from the Trust Index’s implicit calibration target will require per-model threshold adjustment, a consideration that should be treated explicitly in future Trust Index deployments.

Retrieval-depth invariance in NLI-based evaluation. The K sensitivity analysis demonstrates a non-obvious architectural property: detection performance in NLI-based retrieval evaluation is insensitive to retrieval depth for the tested attack strategies. The null result ($K=5 \equiv K=3$) advances the theoretical understanding of where evaluation agent performance is and is not bounded by retrieval quality.

8.2.2 Practical Contributions

A working, reproducible implementation. The complete Trustworthy RAG framework is implemented in Python 3.10 using LangChain, FAISS, HuggingFace Transformers (BART-large-MNLI), and an OpenAI-compatible API for LLM access. The implementation is modular and documented, with independently replaceable components for the retriever, NLI verifier, poison detector, and trust calculator.

Adversarial dataset generator. The *PoisonedDatasetGenerator* implements four rule-based poisoning strategies with configurable poison ratios and supports both TruthfulQA and FEVER benchmarks. It produces reproducible experimental datasets for future comparative studies and is extensible to additional strategies.

Reproducible experiment framework. The experiment runner supports automated multi-model grid evaluation (2×2 factorial design), per-strategy isolated experiments, ablation studies, and retrieval depth sensitivity analysis. Results are automatically serialised to JSON for offline analysis and reproducibility verification.

8.3 Recommendations for Future Research

The findings of this thesis identify seven high-priority directions for future research.

1. Gradient-optimized adversarial evaluation. The four strategies evaluated in this thesis are rule-based approximations of real adversarial attacks. Future work should evaluate the Evaluation Agent against gradient-optimized collision documents of the type described by Zou et al. [4], which are engineered to maximise retrieval similarity while containing arbitrary misinformation. These attacks are likely to evade linguistic pattern detection and may require NLI-adversarial training or embedding-space anomaly detection to identify.

2. Per-LLM Trust Index calibration. The LLM-dependency finding motivates the development of a systematic calibration procedure for the Trust Index threshold τ and component weights α , β , γ . A practical approach would use a small LLM-specific validation set of clean queries to fit τ to the 10th percentile of the clean trust score distribution, or to train a lightweight per-LLM calibration layer that normalises NLI entailment scores for generation-style artefacts.

3. Semantic world-knowledge integration. Entity-swap and subtle manipulation represent the architectural limit of the current surface-signal approach. Integrating external knowledge verification — for example, querying structured knowledge bases such as Wikidata to verify entity-attribute relationships, or employing a search-augmented fact checker for open-domain claims — could substantially improve recall for these attack families. The key design challenge is maintaining acceptable latency while adding an external verification call to each retrieved document.

4. GPU-accelerated evaluation pipeline. Deploying the NLI model on GPU hardware would reduce per-sample evaluation latency from approximately 15 seconds to below 2 seconds, enabling near-real-time RAG evaluation. The current CPU-only implementation is a practical constraint of the FARMI cluster’s resource allocation for student projects, not an inherent architectural limitation.

5. True Attack Success Rate measurement. The current experimental framework measures the Evaluation Agent’s *detection* of poisoned samples, not the LLM’s *adoption* of injected misinformation. Future work should implement ASR measurement via embedding-similarity comparison between generated answers and injected claims, providing a complete end-to-end evaluation of the system’s security value.

6. Retrieval-grounded poisoning labels. The current benchmark protocol uses

document-level index-aligned poisoning labels to determine whether each sample is poisoned. This is appropriate for the controlled paired benchmark setup used in this thesis, but future work should label poisoning based on whether at least one poisoned document is actually retrieved in the query’s top- K set. This would strengthen external validity in open-corpus settings where neighbour retrieval patterns are less deterministic.

7. Multimodal and multi-domain extension. This thesis evaluates the framework on text-only English benchmarks (TruthfulQA and FEVER). Future research should extend the evaluation to domain-specific corpora (medical, legal, scientific), multilingual settings, and multimodal RAG systems incorporating images and structured data, where both the poisoning attack surface and the detection approach may differ substantially.

8.4 Roadmap Toward Secure and Auditable RAG Systems

The Trustworthy RAG framework developed in this thesis represents a first step toward a broader vision: RAG systems that are not merely accurate but *auditable* — systems that provide verifiable evidence for their trustworthiness claims alongside each generated response.

Near-term (1–2 years): The most impactful immediate improvements are GPU-based NLI inference for real-time operation, per-LLM calibration procedures for Trust Index thresholds, and extension to a broader set of LLMs and embedding models. Standardising the Trust Index as a reporting metric in RAG evaluation benchmarks (alongside existing faithfulness and relevance metrics) would help establish it as a community baseline.

Medium-term (2–4 years): Integrating external knowledge verification APIs (structured databases, search engines, specialised fact-checking services) into the evaluation pipeline would directly address the detection ceiling for in-place modification attacks. Streaming evaluation architectures — where the Evaluation Agent operates on individual retrieved passages as they are returned by the retriever rather than on the full batch — would reduce latency and support real-time interactive deployments.

Long-term (4+ years): The ultimate goal is *multi-agent evaluation*: architectures in which specialised agents handle factuality verification, security assessment, consistency checking, and provenance tracking as independent, composable modules. Combined with tamper-evident audit logs and cryptographic provenance chains for retrieved documents, such systems would provide end-to-end accountability for AI-generated information — a critical requirement for deploying generative AI in regulated environments such as health-care, finance, and public administration.

The vulnerability of RAG pipelines to knowledge poisoning is not a temporary imple-

mentation flaw but a structural consequence of the architecture’s reliance on unverified external content. As RAG systems become increasingly central to knowledge-intensive applications, the need for systematic, runtime trustworthiness evaluation will only grow. This thesis establishes that such evaluation is technically feasible, provides a validated framework and implementation, and identifies the specific challenges that must be overcome to make it universally deployable.

8.5 Closing Remark

Knowledge Poisoning turns the core strength of RAG systems — their ability to ground responses in external evidence — into a liability. By demonstrating that an autonomous Evaluation Agent can reliably detect overt poisoning strategies, characterise the architectural limits of surface-signal detection, and produce interpretable trust scores with validated discriminative power, this thesis advances the science of trustworthy generative AI. The path from the current system to one that is universally secure, real-time, and domain-agnostic is long, but the foundation — a principled, empirically validated Trust Index and a multi-signal defence architecture — is now established.

Bibliography

- [1] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [2] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. DOI: 10.1145/3571730. [Online]. Available: <https://dl.acm.org/doi/10.1145/3571730>.
- [3] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [4] W. Zou, R. Geng, B. Wang, and J. Jia, “Poisonedrag: Knowledge poisoning attacks to retrieval-augmented generation of large language models,” in *Proceedings of the 33rd USENIX Security Symposium*, USENIX Association, 2024. [Online]. Available: <https://arxiv.org/abs/2402.07867>.
- [5] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (System Demonstrations)*, 2024, pp. 150–158. [Online]. Available: <https://aclanthology.org/2024.eacl-demo.16/>.
- [6] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, “Ares: An automated evaluation framework for retrieval-augmented generation systems,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024. [Online]. Available: <https://arxiv.org/abs/2311.09476>.
- [7] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>.

- [8] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca9-Abstract-Conference.html.
- [9] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>.
- [10] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hannaneh, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.11511>.
- [11] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>.
- [12] S. Lin, J. Hilton, and O. Evans, “Truthfulqa: Measuring how models mimic human falsehoods,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 3214–3252. [Online]. Available: <https://aclanthology.org/2022.acl-long.229/>.
- [13] O. Honovich et al., “True: Re-evaluating factual consistency evaluation,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*, 2022. [Online]. Available: <https://arxiv.org/abs/2204.04991>.
- [14] K. Greshake, S. Abdelnabi, S. Biller, and C. Wressnegger, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90. [Online]. Available: <https://arxiv.org/abs/2302.12173>.
- [15] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval augmentation reduces hallucination in conversation,” *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 3784–3803, 2021. [Online]. Available: <https://aclanthology.org/2021.findings-emnlp.320/>.
- [16] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “Fever: A large-scale dataset for fact extraction and verification,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics*, 2018. [Online]. Available: <https://aclanthology.org/N18-1074/>.

Appendix A

System Configuration Parameters

Table A.1 lists the complete set of configurable parameters used in the Trustworthy RAG framework, with default values and valid ranges.

Table A.1: System configuration parameters and default values.

Parameter	Default	Range	Description
TOP_K_RETRIEVAL	5	1–10	Number of documents retrieved per query
TRUST_THRESHOLD (τ)	0.5	0.0–1.0	Binary trustworthiness decision boundary
POISON_RATIO	0.30	0.0–1.0	Fraction of poisoned documents in the corpus
α (factuality weight)	0.40	0.0–1.0	Trust Index weight for NLI factuality score
β (consistency weight)	0.35	0.0–1.0	Trust Index weight for inter-document consistency
γ (poison weight)	0.25	0.0–1.0	Trust Index weight for poison safety component
Dampener threshold	0.70	0.5–1.0	P_{poison} value above which the non-linear dampener activates
Dampener max reduction	0.40	0.0–1.0	Maximum multiplicative trust score reduction at $P_{\text{poison}} = 1.0$
FAISS confidence floor	0.30	0.0–1.0	Minimum retrieval similarity score before a trust penalty is applied
NLI entailment threshold	0.40	0.0–1.0	Minimum entailment score for a document to contribute to factuality
Cross-doc contradiction threshold	0.70	0.5–1.0	Threshold for high-severity cross-document contradiction signal
Intra-doc contradiction threshold	0.70	0.5–1.0	Threshold for intra-document first/second half NLI contradiction
NLI_MODEL	facebook/bart-large-mnli		NLI model used for factuality and poison detection
EMBEDDING_MODEL	all-MiniLM-L6-v2		Sentence embedding model for vector retrieval
LLM_MODEL	llama3.3:70b		Primary LLM used for response generation

Appendix B

Extended Experimental Results

This appendix provides supplementary results tables referenced in Chapter 6.

B.1 Per-Strategy Results — All Metrics

Table B.1 extends the per-strategy summary with full TP/FP/FN counts and trust score separation values (100 samples per strategy, TruthfulQA, $K = 5$).

Table B.1: Per-strategy detection results (Llama 3.3 70B + all-MiniLM-L6-v2, 100 samples each, TruthfulQA, $K = 5$). Results from isolated single-strategy experiments; mixed-strategy results reported in Chapter 6.

Strategy	Acc.	Prec.	Recall	F1	Sep.	TP	FP	FN
Contradiction	92%	88.9%	53.3%	66.7%	0.311	8	1	7
Instruction Injection	99%	93.8%	100%	96.8%	0.498	15	1	0
Entity Swap	85%	—	0%	0%	0.053	0	0	15
Subtle	88%	100%	20.0%	33.3%	0.149	3	0	12
Mixed	91%	100%	40.0%	57.1%	0.225	6	0	9

B.2 Ablation Study Results

Table B.2 reports the full ablation study results across five Trust Index weight configurations.

B.3 FEVER Benchmark Pilot Results

Table B.3 reports early pilot results on the FEVER benchmark (Llama 3.3 70B + all-MiniLM-L6-v2). These 20-sample pilot results indicated high accuracy (90%) initially, but a subsequent full 100-sample experiment (reported in Chapter 6) revealed broader generalisability challenges.

Table B.2: Ablation study: Trust Index weight configurations and detection performance (TruthfulQA, 60 samples).

Configuration	α	β	γ	Acc.	Prec.	Recall	F1
Default	0.40	0.35	0.25	86.7%	100%	11.1%	20.0%
Equal weights	0.33	0.34	0.33	86.7%	100%	11.1%	20.0%
Factuality-heavy	0.60	0.20	0.20	86.7%	100%	11.1%	20.0%
Consistency-heavy	0.20	0.60	0.20	88.3%	100%	22.2%	36.4%
Poison-heavy	0.20	0.20	0.60	88.3%	100%	22.2%	36.4%

Table B.3: FEVER benchmark early pilot results after KB enrichment fix (20 samples, mixed strategy, $K = 3$).

Metric	Value	Metric	Value
Accuracy	90%	Clean Trust $\bar{T}$	0.654
Precision	—	Poison Trust $\bar{T}$	—
Recall	66.7%	Separation	—
F1	—	vs. Baseline	+5%

Appendix C

LLM Prompt Templates

This appendix lists the prompt templates used in the RAG pipeline for response generation and the system prompt governing the LLM’s behaviour.

C.1 RAG Generation Prompt

The following template is used by the generator to produce responses from retrieved context. The placeholders `{context}` and `{question}` are filled at runtime by the pipeline orchestrator.

```
You are a helpful assistant. Answer the question based ONLY on the
provided context. If the context does not contain enough information
to answer the question, say "I don't have enough information to
answer this question."
```

```
Context:
{context}
```

```
Question: {question}
```

```
Answer:
```

C.2 Design Rationale

The prompt is intentionally minimal and instruction-focused. The phrase “based ONLY on the provided context” is included to maximise the LLM’s reliance on retrieved documents (promoting faithfulness) and to reduce parametric knowledge injection (which would undermine the retrieval-grounded evaluation). The explicit fallback instruction (“I don’t have enough information...”) prevents the model from hallucinating answers when

retrieved context is insufficient, which would otherwise produce low NLI entailment scores and artificially depressed Trust Index values.

No chain-of-thought or few-shot examples are included, consistent with the evaluation design principle that the generation step should reflect a standard, unaugmented RAG deployment rather than a reasoning-enhanced configuration.

Appendix D

Ethical Considerations and Responsible Research Statement

D.1 Research Ethics Declaration

This thesis conducts research on adversarial attacks and defences in AI systems. All experiments were performed in a controlled, isolated research environment using publicly available benchmark datasets (TruthfulQA and FEVER). No personally identifiable information was collected, processed, or stored at any stage of the research.

The adversarial dataset generator and poisoning strategies implemented in this thesis are designed exclusively for the purpose of evaluating the Evaluation Agent’s detection capabilities. They are not intended to serve as operational attack tools. The strategies are rule-based transformations (not gradient-optimized collision attacks) and do not provide meaningful uplift to malicious actors beyond what is already published in the open literature [4, 14].

D.2 Dual-Use Considerations

The detection techniques developed in this thesis — particularly the linguistic pattern library and the intra-document NLI approach — could theoretically inform adversaries about the specific features monitored by the detector. This dual-use risk is inherent to all published security research and is mitigated by the transparent documentation of both the detection system’s capabilities and its known limitations (particularly the near-zero recall for entity-swap and subtle manipulation strategies). The publication of defensive limitations serves the broader research community by directing future work toward the correct problem: detection of in-place semantic modifications using external world-knowledge, rather than surface-level textual signals.

D.3 Data Handling

All datasets used in this research are publicly licensed:

- **TruthfulQA** [12]: Apache License 2.0
- **FEVER** [16]: Creative Commons Attribution-ShareAlike 3.0 Unported

The synthetic poisoned datasets generated during experiments are derived from these sources and carry the same license restrictions. No proprietary, confidential, or sensitive data sources were used.

D.4 AI-Assisted Research Tools

Portions of the implementation code and thesis writing process benefited from AI-assisted development tools (Claude Code, Anthropic). All AI-generated content was reviewed, validated, and edited by the thesis author. The experimental results, analysis, and conclusions are the independent work of the author, verified against the actual codebase and experiment outputs.

Appendix E

Implementation Details and Repository Structure

E.1 Repository Structure

The project repository is organised as follows:

```
RAG Agent/  
src/  
  retriever/  
    embeddings.py      # Embedding model wrappers (local + API)  
    retriever.py       # FAISS-based document retriever  
  evaluation_agent/  
    nli_verifier.py    # BART-MNLI NLI inference  
    poison_detector.py # Five-signal poison detection  
    trust_index.py     # Trust Index computation + dampener  
    evaluation_agent.py # Orchestration class  
  generator/  
    llm_generator.py   # OpenAI-compatible LLM interface  
  pipeline/  
    rag_pipeline.py    # End-to-end RAG pipeline  
    experiment_runner.py # Experiment execution + metrics  
src/experiments/  
  poisoned_dataset.py  # Adversarial dataset generator  
configs/  
  config.yaml          # Master configuration file  
data/  
  raw/                 # Downloaded benchmark datasets  
  processed/           # Preprocessed knowledge bases
```

```

    experiments/          # Serialised experiment results (JSON)
Master_Thesis/
    main.tex             # LaTeX document root
    chapters/           # Chapter and appendix .tex files
run_experiment.py        # CLI experiment runner
requirements.txt         # Python package dependencies

```

E.2 Key Dependencies

Table E.1: Core Python dependencies and versions.

Package	Version	Purpose
Python	3.10+	Runtime language
LangChain	0.1+	Pipeline orchestration
faiss-cpu	1.7+	Vector similarity search
transformers	4.35+	BART-MNLI NLI model
sentence-transformers	2.2+	MiniLM-L6-v2 embeddings
torch	2.0+	NLI model inference (CPU)
openai	1.0+	FARMI API client
datasets	2.14+	HuggingFace dataset loading
loguru	0.7+	Structured logging

E.3 Reproduction Instructions

To reproduce the primary experiment results (Llama 3.3 70B + all-MiniLM-L6-v2, TruthfulQA, 100 samples, mixed strategy, K=5):

1. Install dependencies: `pip install -r requirements.txt`
2. Configure FARMI API credentials in `.env`: set `FARMI_API_KEY` and `FARMI_API_BASE`
3. Run: `python run_experiment.py`; at the interactive prompt, select Llama 3.3 70B, all-MiniLM-L6-v2, and K=5
4. Results are saved to `data/experiments/` as a timestamped JSON file

To reproduce the full 2×2 grid (K=3):

```
python run_experiment.py --grid --samples 50
```

To reproduce the K=5 sensitivity analysis:

```
python run_experiment.py --grid --samples 50
```

```
# Select K=5 at the interactive prompt, or set TOP_K_RETRIEVAL=5 in .env
```

All experiment results used in this thesis are archived in `data/experiments/` with ISO 8601 timestamps in the filename. The primary results file is:

```
experiment_truthfulqa_llama3.3-70b_all-MiniLM-L6-v2_detection_20260307_112613.json
```